

PRE-PRODUCTION · PREVIEW EDITION

How the Machine Thinks

A technical architecture reference for the NestVM epistemic workbench — the wave log, the analytical constructs built on it, and the delegated Cell that does its controlled thinking.

Document reference

CoAgnes / NestVM / ARCH-PREVIEW /
v0.9

Edition date

15 September
2026

Status

Draft · not for external
distribution

Document Control

Title	How the Machine Thinks – NestVM Technical Architecture Reference
Document reference	CoAgnes / NestVM / ARCH-PREVIEW / v0.9
Status	Pre-production · Preview edition · Draft
Edition date	15 September 2026
Prepared by	CoAgnes NestVM documentation, drafted with Claude (Anthropic)
Prepared for	Internal review and familiarisation – engineering and analytical stakeholders
Language	English (British)
Source material	The book_p09 reference vault and the pr09 session walkthrough, CoAgnes NestVM assembly (coagnes/nestvm/)

Version	Date	Author	Description
0.9	15 September 2026	CoAgnes NestVM documentation	First assembled preview edition, compiled from the seven-part technical reference and the worked session record.

PRE-PRODUCTION PREVIEW – SUBJECT TO CHANGE

This edition is an introductory, familiarisation-stage document. It describes the system as implemented and verified against a live session at the time of writing, but the specification, the surface, and the terminology it documents are all still under active revision. Nothing in this edition should be treated as a frozen interface contract. Page and section references are internal to this document only.

Executive Summary

This reference describes NestVM as assembled inside CoAgnes: a machine whose entire behaviour is built from one append-only log and two kinds of reactive component – a listener that waits for a declared shape of fact, and a doer that fires once everything it needs has arrived. Every capability documented here, from a single form on screen to a fully delegated, human-attested sub-investigation, is this same pair, composed.

The document is organised in seven parts, each covering one facet of the system from four consistent angles – a plain-language overview, the standard analytical practice it mirrors, the machine's own functional specification, and the concrete technical implementation. Part 4 closes with a fully worked

example: one real press-release study, walked from the opening intention to the finished, human-reviewed document, with every offset and every function on its path independently verifiable against the stored session log.

Readers new to the system should read Parts 1 and 3 first to establish the two working ingredients; readers principally concerned with the analytical workflow should proceed to Part 4; readers integrating against the system should read Part 5 in full. Part 7, on the Cell, is this edition's most complete treatment and is the recommended entry point for anyone evaluating how the system handles delegated, model-assisted work under human review.

Table of Contents

01 System Overview	5
1.1 Overview	5
1.2 Analytical Rationale	6
1.3 Functional Specification	7
1.4 Technical Implementation	9
02 Lines of Reasoning	12
2.1 Overview	12
2.2 Analytical Rationale	13
2.3 Functional Specification	15
2.4 Technical Implementation	17
03 The Minimal Fragment	20
3.1 Overview	20
3.2 Analytical Rationale	21
3.3 Functional Specification	23
3.4 Technical Implementation	25
04 The Standard Analytical Scenario	28
4.1 Overview	28
4.2 Analytical Rationale	29
4.3 Functional Specification	30
4.4 Technical Implementation	33
4.5 Worked Reference Implementation	34
05 Protocols and Integration Points	39
5.1 Overview	39
5.2 Analytical Rationale	40

5.3 Functional Specification	41
5.4 Technical Implementation	45
06 System Architecture and Component Interconnection	51
6.1 Overview	51
6.2 Analytical Rationale	52
6.3 Functional Specification	53
6.4 Technical Implementation	57
07 The Cell – A Local Field of Meaning	60
7.1 Overview	60
7.2 Analytical Rationale	61
7.3 Functional Specification	63
7.4 Technical Implementation	66
Appendix A – Glossary	69
Appendix B – Sources and References	75
List of Figures	
Figure 4.1 – The workbench sheet at the close of the worked scenario	37

PART 1 OF 7 • SYSTEM OVERVIEW

CHAPTER 01

System Overview

How the machine works, seen as a whole — the log, listeners and doers, and the settle loop that drives them.

1.1 Overview

SCOPE What are we doing here.

Everything is one list

Imagine a single, ever-growing list. Every entry on it is small — a handful of fields — and once written, never changes and never moves. That list is the whole of what the system knows: not a summary of it, not a cache of it — it is it. Nothing the system ever shows you exists anywhere else.

Now put two kinds of small machines around that list.

Listeners. A listener watches the list for entries of a particular shape — "tell me whenever something matching this arrives." It keeps a tiny running note of what it has seen so far (sometimes just the latest one, sometimes everything so far), and when its note finally has everything it was waiting for, it says so — by writing one more entry onto the very same list, announcing "I'm ready, and here is what I collected."

Doers. A doer watches for exactly that "I'm ready" announcement from one or more listeners. Once every listener it cares about has announced readiness, it does its one thing — write something, ask a question of a language model, hand off to a person — and whatever it produces becomes, again, more entries on the list.

That is the entire machine. There is no third kind of thing. A person typing an answer, a report being synthesised from four separate readings, a whole delegated sub-investigation opening and closing — every one of them is built from nothing but listeners and doers, arranged and re-arranged, watching the same one list.

Why "the list never changes" matters more than it sounds like it should

Because it means the system's entire history is always available, in order, to anyone who wants to look — and because nothing can quietly go missing or get rewritten, every later claim can, in principle, be traced back to the exact moment it entered. This single property is what lets the rest of this book talk confidently about "where a finding came from" ([Part 7](#)) and "what the machine actually did" ([Part 6](#)) without hand-waving — the answer to both is always, literally, "look at the list."

Where a person fits

A person interacts with the system entirely by adding entries to the same list — typing an answer, pressing a button that means "yes, use this," pasting in a question. Nothing a person does is treated specially by the machinery underneath; a person's contribution is just another entry, distinguishable from a machine's only by what it says, never by some separate privileged channel. What the person sees on screen, in turn, is never anything more than a live read of the list, redrawn — never a second source of truth living only in a browser tab.

The shape that repeats at every scale

The same rhythm — something arrives, a listener collects until it has enough, it announces readiness, a doer acts, more arrives — repeats identically whether you are looking at one tiny form on screen or an entire delegated sub-investigation running for a minute unattended. [Part 7](#) is exactly this same shape, once more, wrapped in its own small fence so its findings do not leak out before a person has looked at them. Nothing bigger the system does is a different kind of thing from a form waiting for one field to be filled in — it is only ever more of the same two ingredients, composed.

1.2 Analytical Rationale

METHOD How do we define and fill systemic gaps.

The gap most tools quietly carry

Ask most software "what actually happened here, in order, and why" and the honest answer is usually reconstructed after the fact — from server logs meant for debugging, from a database's current state with its history discarded, from a chat transcript that mixes the system's reasoning with its user-facing text. The system was never designed to answer that question; the answer is scavenged.

1.1's one list is a direct answer to that gap, made structural rather than scavenged: **the record of what happened is the system**, not a side effect of it. There is nothing to reconstruct because nothing was ever thrown away or summarised in the first place.

Three standard analytical needs, met by construction

Provenance. Every finding a study produces can be traced to the specific entries that fed it — not "somewhere in this conversation," but a named position in an ordered list, forever. This is the same discipline a court exhibit, an audit workpaper, or a cited fact in a piece of journalism is held to: show your working, all the way down.

Reproducibility. Because nothing is ever edited or removed, replaying the same list from the start reconstructs the exact same state every time — which is what lets [Part 4](#)'s worked example be checked by a second reader independently rather than taken on trust, and what lets [Part 6](#)'s "readings" be thrown away and rebuilt at will with no loss.

A clean seam between drafting and deciding. Because listeners and doers only ever react — nothing acts on its own initiative outside what it was told to watch for — a human decision (accept this, attest that) is always a distinct, visible entry of its own, never folded silently into a machine step. [7.2](#) develops this into the specific shape of a delegated sub-task with a sign-off; here, it is worth naming as the general property underneath it: **the list can always tell you which entries a person actually decided, and which ones a machine merely produced.**

What this trades away, honestly

A system built this way is not the fastest possible way to get an answer, and it is not free-form — you cannot simply "chat" your way to a result outside the shape of listeners waiting for specific things. [Part 3](#) takes this trade-off head-on: the cost of every entry having a fixed, checkable shape is that nothing enters the list informally, and building a new capability means declaring, in advance, exactly what it will wait for and what it will do once it sees it. The next file, [1.3](#), gives that discipline its formal names.

1.3 Functional Specification

SPECIFICATION Where and how flows form, in the machine's own vocabulary.

Source: `lib/core/wave/`, `lib/core/topology/`, `lib/core/runtime/`, `Lib/TOPOLGY.md`
(NestVM Vol. 02–08, incubated in this repository).

The list is the log; an entry is a tuple

The "list" of 1.1 is the **wave log** — append-only, dense (no gaps), never edited. One entry is a **tuple**, and it always has exactly four fields:

```
type WaveTuple = Readonly<{
  offset: number;           // its position, assigned only by the engine
  kind: string;             // one of four fixed kinds — see below
  key: string | null;       // which lane it travels on; null = lane-less
  payload: unknown;        // what it actually says
}>;
```

There are exactly **four kinds** of tuple, and nothing else: `sys.knot.defined` and `sys.descriptor.defined` register a listener or a doer (this is authoring, covered fully in Part 3); `domain.fact` is an ordinary committed fact — a person's words, a machine's answer, anything of substance; `sys.knot.ready` is a listener's own announcement that it has everything it needs. A record submitted for the log without its `offset` yet is called an **emission**; the engine assigns the offset the instant it actually appends.

Listeners are knots, doers are binds

A **knot** is the listener of 1.1. Its behaviour is exactly five small operations, and every knot in the system — however complicated the study built from it looks — is only ever these five: does this tuple match what I collect (`matches`); if so, fold it into what I've collected (`wind`); has what I've collected now satisfied my condition (`test`); if so, what exactly did I collect (`understanding` — this becomes the readiness announcement's own content); and once I've announced, do I keep what I collected or start clean (`reset`).

A **bind** is the doer. The specification names two shapes for it: an **operator bind** waits for one or more knots' readiness (its demands) and, once every one of them has fired at least once since it last acted, does its one thing — exactly the "barrier" behaviour named in Part 2. An **emit descriptor** is simpler still: it has no demands at all, and republishes a single knot's readiness, verbatim, as a fact of a declared type, every single time that knot fires.

The rhythm of one wave

One full pass — the specification's word is a **wave** — is: take the next unprocessed tuple; hand it to the machine's one processor; if any listener's condition is now satisfied, announce its readiness (itself a new tuple, appended immediately, seen by the very next step of the same pass); if any doer's barrier is now satisfied, let it act (again, new tuples, appended immediately); keep going until nothing is left to

process. That final state — nothing left, no doer waiting on an outside answer either — is called **quiescence**, and a wave is only ever considered finished once quiescence is reached, never partway through.

Crossing to the outside world

Some doers need something the machine itself cannot supply — a language model's answer, most often. For that there is exactly one crossing point, the **membrane**: a second, independent reader of the very same log, offering each tuple once to every registered **controller** (a language-model connection, a deterministic stand-in, and so on). A controller that recognises a tuple as its own kind of request answers it asynchronously; when that answer arrives, it re-enters the log as new tuples, and the wave continues from there — so a delegated question and its eventual answer are still, in the end, just two more entries on the one list, correlated by an id carried in both, never by which one happened to come right after the other (the specification is explicit that position in the log is never itself evidence of a causal link — 2.3 returns to exactly why).

1.4 Technical Implementation

IMPLEMENTATION How exactly is this implemented at the program level, and how the system checks its own consistency.

Source: `lib/core/wave/create-wave-engine.ts`, `lib/core/runtime/create-wave-runtime.ts`, `lib/core/membrane/create-membrane.ts`, `lib/core/registers/*`.

The register file — the machine's whole state, as one value

Everything the machine is doing right now — where its two cursors are, what every listener has collected so far, which outside answers are still pending — lives in exactly one value, the **register file**. Nothing is held in a private variable anywhere else. Every single change to it goes through one function, `recordTransition`, which — in the same movement — writes the change and appends a line to a **journal** naming which function made it, from which file, under which tuple's context. This journal is not a debugging afterthought; it is how Part 6.4 can trace a gesture through every ring with total confidence, and how Part 7.4 can show exactly which check ran when.

The two cursors that make "exactly once" true

Two independent counters walk the same log, never sharing a position, never moving backwards:

- `takeNextTuple` advances the engine's own cursor and hands the tuple to every listener and doer inside the machine — this is the "hand it to every processor" step of 1.3's rhythm.

- `sweepNextTuple` advances the membrane's cursor, entirely independently, and offers the same tuple to every outside controller.

Because each cursor only ever moves forward by exactly one, a tuple is seen by the inside machinery exactly once and offered to the outside world exactly once — the specification's exactly-once discharge guarantee is not a promise kept by convention; it is a direct consequence of there being only one way to advance either counter.

The settle loop — alternation until quiescence

```
runtime.settle:
  repeat:
    engine.run over the current pending records      # the "wave" of 1.3
    membrane.sweep — offer every new tuple to every controller
    if nothing is waiting on an outside answer: done — return the log
    otherwise: wait for the EARLIEST outside answer to arrive,
                feed its records back in as the next pending batch,
                go around again
```

This is the whole of `createWaveRuntime.settle` — under twenty lines, and it is the only place the machine ever waits for the outside world. Every request the system handles, however large the chain of consequences turns out to be, is exactly one call to this loop, run once, to completion.

A trace small enough to hold in your head

The very first wave of any session — the moment a person's opening question is committed — is a compact instance of every piece above, drawn from a real session (`loop-sem-1` , walked in full in the sister vault):

offset	kind	what happened
#0-#1	<code>sys.knot.defined</code> , <code>sys.descriptor.defined</code>	the front door's one listener and one doer are registered — the log's very first two entries
#2	<code>domain.fact (intent.given)</code>	the person's question is committed
#4	<code>sys.knot.ready</code>	the listener's condition is met on the first fact it was ever offered — it announces readiness, carrying the question
#5	<code>domain.fact (service.request)</code>	the doer, activated by that readiness, asks the outside world which study method fits

offset	kind	what happened
—	(the membrane sweeps #5; a controller claims it; the machine now waits)	
#6	<code>domain.fact (programme.chosen)</code>	the controller's answer re-enters as a new tuple, correlated to #5 by a shared id, not by simply following it in the log
#7-#16	registration + a final fact	a second controller, seeing #6, registers a whole study method's worth of listeners and does in one go, and announces it has done so

Seventeen tuples, one wave, one call to `settle`, no gaps in the numbering — and every arrow in that table is a real journal line, not an inference. The companion technical reference reproduces, tuple by tuple, every one of these tuples' exact payloads and every journal transition behind them.

How the machine checks itself

Because every state change is journalled the same way, the codebase can and does hold itself to mechanical proof rather than only careful writing: a **golden fixture** test replays a fixed script and checks the resulting log byte-for-byte against a known-good recording; a **parity oracle** runs the identical script two ways — continuously, and via a snapshot-and-restore in the middle — and checks the two runs are indistinguishable; the **ring-discipline** guard (met properly in [Part 6](#)) scans every file's imports to confirm no ring reached past its neighbour. None of this proves any one finding is correct — that is always a human's job, taken up fully in [Part 7](#) — but it proves the machinery that produced it did not silently misbehave on the way.

Lines of Reasoning

What forms a chain of analytical steps inside the system, and the one mechanism — represent — that carries a finding across any boundary.

2.1 Overview

SCOPE What are we doing here.

Chains are not a separate feature

There is no button marked "start a line of reasoning," and no construct called "argument" or "chain" anywhere in the machine's vocabulary. A chain is simply what you get, for free, the moment one doer's published output happens to be the exact fact type a different listener was already waiting for. Nothing has to be told this is a chain — the machine has no idea, and does not need to; it only ever asks each listener, independently, "has what you were waiting for arrived," and each doer, independently, "has everything you demand fired." A line of reasoning is a pattern a reader sees in the wiring, not a thing the machine tracks as such.

A seven-step chain, walked once, plainly

One of the system's own study designs makes an unusually clean seven-step chain visible: a person refines their own understanding through seven named levels — what am I seeking to understand (L1); where does it apply and where does it not (L2); how do the parts depend on each other (L3); what shape must the answer take (L4); into what pieces does it break down (L5); what changes and what stays stable (L6); and, finally, does the whole answer serve what was actually asked, checked against the person's own response (L7). Each level's listener waits for exactly two things: the person's own refinement at that level, and the previous level's published output, arriving as a fact of a type named `knowledge.L<n-1>` — nothing before both are present, no reordering possible. The output of L6 becomes, unmodified in kind, the very fact L7's own listener is watching for. Seven pairs, wired so each one's condition names the last one's result — and the whole thing reads, from a distance, exactly like a structured argument advancing level by level, because that is precisely what it mechanically is.

Chains do not have to be a straight line

The four-question study behind most of this book's worked examples is not a single chain at all — it is **four separate, independent lines running side by side**, each opening its own delegated sub-investigation, each releasing its own finding, and a fifth listener waiting for all four before it does anything: the report. Nothing forces a line of reasoning to be sequential. A listener's condition can just as easily name several unrelated things at once, and the doer attached to it will not fire until every one of them has arrived — which is the machine's native way of expressing "do not synthesise until every branch of the investigation is actually in."

Where a delegated sub-investigation fits into a chain

Part 7's Cell is not a break in a line of reasoning — it is a segment of one, fenced off and returned. A chain can perfectly well have one of its steps be "hand this off, wait for a person to sign off on what came back, and only then continue" — and the return, once it happens, re-enters the chain exactly the same way any other step's output would: as a fact of a declared type, landing in the listener that was waiting for it all along.

Where knowledge actually crosses a boundary

Every one of the examples above shares one detail worth naming up front, because 2.3 and 2.4 build the rest of the part around it: a finding never simply appears inside a new listener's slot by magic. Something has to carry it there — the person's own intention, re-entering a delegated Cell as its starting material; a Cell's accepted export, carried back into the exact slot that asked for it; a released finding, cited again inside a later step without being retyped by hand. That one carrying mechanism, used identically every single time knowledge needs to cross from where it was established to where it is needed next, is what the rest of this part is actually about.

2.2 Analytical Rationale

METHOD How do we define and fill systemic gaps — what standard research method already knows about building an argument, and how the machine mirrors it exactly.

Three moves every serious piece of analysis makes

Decompose before you synthesise. A hard question is rarely answered directly; it is broken into narrower sub-questions that can each be settled on their own evidence, and only combined once every one of them has an answer. 2.1's four parallel vectors are this move, mechanised: four independent lines, four independent Cells, four independent findings – and a fifth step, the report, that is structurally incapable of firing until all four exist, because its listener's condition names all four by type.

Do not write the synthesis early. This is the barrier, and it is worth stating as its own discipline because it is so easy to violate informally – a person drafting a report by hand will often start the conclusion before every input has actually arrived, then quietly patch it later. A listener that demands several things at once cannot do this even if asked to; there is no partial firing, no draft state visible to anything downstream. The report either does not exist yet, or it exists having genuinely seen everything it was supposed to.

Cite a prior finding; never silently re-derive it. Good analytical writing distinguishes sharply between reaching a new conclusion and reusing an established one – a citation names its source and carries the finding forward unchanged, rather than restating it from memory and risking drift. This is exactly what 2.1's "carrying mechanism" does mechanically: a finding that re-enters a later step arrives stamped with exactly where it came from, and its content is never retyped by a person or reconstructed by a machine – it is the same value, moved.

Why this matters more, not less, once a language model is involved

A language model will, if asked, happily restate an earlier finding from its own memory of the conversation – fluently, and not always accurately. The discipline above closes that gap structurally: nothing in this system ever asks a model to recall an earlier finding; every reuse of a finding is a mechanical carry of the actual committed value, and a model is only ever asked to produce something new from material it was explicitly handed. The distinction between "the model derived this" and "the model was shown this, unchanged, from elsewhere" is never blurred, because the machine has no path by which it could be.

Provenance is not just where a fact came from – it is also how it got there

Ordinary provenance answers "who said this, and when." The mechanism this part is building towards answers a second question research method cares about just as much: by what path did a finding travel from where it was established to where it is now being used – copied verbatim, retitled and re-scoped for a new audience, or handed down into a stricter, walled sub-task with a narrower brief

attached? 2.3 gives this its formal shape: **answer**, **attach**, and **citation** are three genuinely different kinds of carry, and the machine keeps the difference visible rather than collapsing all reuse into one undifferentiated "referenced elsewhere."

What this does not solve

None of this checks whether a finding is actually right — only that its path from origin to reuse is unbroken and honestly labelled. Whether the finding itself deserves to be trusted is, again, the analyst's own job, taken up fully in 7.2. What this part's mechanism buys is narrower and, for exactly that reason, dependable: if a claim is wrong, you can always find out where it was produced and how it got to where you are reading it — nothing about the reuse itself can have quietly changed it along the way.

2.3 Functional Specification

SPECIFICATION Where and how flows form, and precisely where the key vectors that move knowledge between planes are declared.

Source: [spec/PROGRAMME_MANUAL.md](#) (chapters 6, 10, 11), [nestvm/programmes/epistemic-cascade.yaml](#), [nestvm/lib/API.md](#) §5, [CLAUDE.md](#) (host, "Thinking rails").

Rail 5 — a return socket is declared at birth, never improvised

A listener may only ever be told, at the moment it is registered, which of its own slots a later delegated answer is allowed to fill (`config.meta.return`, naming one of its collection rules). This is not a convenience — it is a hard rule of the underlying specification (Vol. 02 §5.1): nothing committed before a listener existed can ever reach it, so there is no way to wire a return after the fact. Concretely, in the worked study of Part 4, the vector listener declares its return before any question has even been typed:

```
id: press.facts.wanted
config:
  meta: { return: findings }
  wind:
    collect:
      - { as: question, match_type: press.vector.given, reduce: latest, where: [{field:
vector, equals: facts}] }
      - { as: findings, match_type: press.findings.given, reduce: latest, where: [{field:
vector, equals: facts}] }
    condition: "question ≠ null && findings ≠ null"
```

When a Cell is later opened from this exact slot, the machine already knows — because it was told at registration, not guessed afterwards — that whatever comes back must arrive as a `press.findings.given` fact carrying the `facts` discriminator. Nothing about opening the Cell has to re-specify where its answer belongs; the address was fixed the moment the listener was born.

Rail 4 — represent, the one reuse move, used for every kind of carry

Rail 4 is the specification's name for the single mechanism behind everything 2.1 called "carrying": a committed value re-entering a different declared hearing, as a fresh fact, without ever being retyped. It has exactly three request shapes, and the manual (chapter 11, "the informational patterns") is precise about keeping them distinct rather than collapsing them into one idea of "reuse":

- **answer** — the value fills the hearing's own field directly, as if the person had typed it themselves;
- **attach** — the value rides alongside a person's own note, as a named attachment, never silently replacing what the person wrote;
- **citation** — a released value is pointed at again from a new location, carrying a stamp naming exactly where it originally came from, rather than being copied as if freshly stated.

Every one of these answers, on the log, is stamped `represented`, naming its source by session and offset — so a reader can always tell a value's current location from its point of origin, however many times it has been carried since.

Rail 6 — authoring a line of reasoning is heavier than adding to one

Committing a fact is light: it is one entry, on an existing lane, judged by rules that already exist. **Opening a Cell — descending into a new branch of the reasoning — is categorically heavier**, because it registers new topology: new listeners, new doers, an entire admission ceremony behind them, all before the first fact reaches any of it. The specification names this explicitly as its own rail, deliberately distinguished so that the surface never lets the two actions look alike — Part 4's worked walkthrough shows this as a visibly heavier gesture (a gold, "authoring weight" control) than the ordinary grey commit button beside it, and that visual difference is a direct, deliberate reflection of this rail, not a styling choice made independently of it.

The seven-level cascade's own two extra wrinkles

Two details in `epistemic-cascade.yaml` are worth naming because they show the chain of 2.1 is not perfectly rigid:

- **Level 3 has a side channel.** Alongside its ordinary two demands (the person's own refinement, the previous level's output), L3 also listens for `addressed.given` facts pointed at it by name, accumulating them (`reduce: "append"`) as running annotations rather than requiring exactly

one. A chain, in other words, can have a step that keeps gathering rather than only ever waiting for one fixed thing.

- **Level 7 is not a plain deterministic gate at all.** Its listener runs under a different strategy entirely — `semantic_evaluator` — whose condition is not a null-guard but a genuinely semantic one: two explicit questions ("does the gathered answer serve what was sought," "do the control questions hold against the whole") judged against a minimum grade. And even once that semantic bar is cleared, L7 still will not seal until a second, entirely separate fact arrives — the person's own response — so the chain's very last link is deliberately built to require both a machine judgement and a human one before it closes.

Beyond one session — the same move, at a larger scale

Nothing about a line of reasoning is confined to one session. A fresh session's opening fact may itself declare a `parent` — the exact slot, in an entirely different session, that it exists to answer — and a **lineage** reading walks this chain of parents upward to a root and downward to every child, entirely by reading each session's own opening fact by address, never by any separate index maintained on the side. This is rail 5's return socket, generalised one level up: a whole session can be a delegated branch of a larger line of reasoning, exactly as a Cell is a delegated branch of a single session's.

2.4 Technical Implementation

IMPLEMENTATION How exactly is this implemented at the program level, and where the key vectors between planes actually run in code.

Source: `lib/application/act-deepen.ts`, `lib/side/world/create-represent-controller.ts`, `lib/application/read-lineage.ts`, `lib/application/log-address.ts`.

Sowing a return, in two ingests, atomically

Opening a Cell from a declared slot is, underneath, exactly **two write requests treated as one act** — and the split is deliberate, not incidental. The first request runs the whole admission ceremony (7.3); only if that request's own receipt actually contains `cell.opened` does the act go on to the second request: sowing, at the root, a fresh listener whose two demands are the Cell's own accepted-export type (discriminated by the new Cell's own reference) and a `return.declared` fact naming the exact slot to carry it into — followed immediately by an emit descriptor that republishes that listener's readiness as a request to the **represent** controller. If the first request is refused, the second is never sent at all — nothing half-registers a return address for a Cell that does not exist.

```
// lib/application/act-deepen.ts, condensed
const admitted = await actAdmitBlueprint(app, session, { blueprint, accepts, charter,
id }); // request one
if (!admitted.opened) throw new Error(/* admission was refused; stop here */);
await ingestAct(app, session, [
  knotDefined("return.cell-N.gate", { meta: { returnOf: { cellRef, slot } },
    wind: { collect: [ { as: "export", match_type: acceptedType, where: [{ field: "cellRef",
equals: cellRef } ] },
      { as: "address", match_type: "return.declared", where: [{ field:
"cellRef", equals: cellRef } ] } ] },
    condition: "export ≠ null && address ≠ null" }),
  descriptorDefined("return.cell-N", { subscribesTo: "return.cell-N.gate", actionConfig:
{ writes: "represent.requested" } }),
  fact("return.declared", { cellRef, slot, as: theSlotsOwnHearing, exportType: acceptedType })
]); // request two
```

The carrier, precisely

createRepresentController answers **represent.requested** facts through two code paths that collapse three request shapes into them. A **return** request — recognisable by carrying both an **address** and an **export** together — takes the export's own result and places it into the address's declared field as it is: a plain string stays a string, a structured finding stays a structure, never flattened into encoded text along the way. A **person's** request — **{from: {session, offset}, as: {factType, field, where}, mode}** — reads the named tuple straight from the log **by its address**, never through any computed reading, checks that it is an ordinary, open fact (a Cell's still-private material, or anything under a reserved name, is refused outright), and places its value the same way. A **bind's delegated** request is not a third code path at all — it unwraps into the very same handling as the person's own, because underneath it is asking for exactly the same thing on someone else's behalf. Every answer, whichever path produced it, is stamped **represented** with its origin, and correlated to its own request by an id — never by simply sitting next to it in the log.

Tracking a chain across whole sessions

readLineage answers "what is this session's place in a larger line of reasoning" by reading, not maintaining. **intentOf** fetches a session's first handful of positions **by address** — never by loading and folding the whole session — until it finds the opening **intent.given** fact and, if present, the **parent {session, bind, knot, hearing}** it carries; **readLineage** then walks that chain of parents upward to a root, and separately scans every stored session for one whose own **parent.session** names the session in question, to find its children. Nothing about a lineage is a separate index kept in step with the log — it is recomputed, cheaply, from the log itself, every single time it is asked for.

Cheaply is worth a word of honesty: the first version of this function read each ancestor's **whole** history to find one fact near the start of it, and on a shelf of many sessions that made the lineage reading slow enough to visibly stall a busy screen — a defect found and fixed, mid-project, precisely by reading by address instead. The fix changed nothing about what the reading returns; it only stopped it from doing far more work than the question actually required — a small, concrete instance of [Part 6.2's](#) point that a reading is only as trustworthy as it is honest about the cost of asking it.

One return, worked from real offsets

Cell `cell@37`'s return, from the same worked session as [Part 7.4](#), seen this time through the carrying mechanism rather than the Cell's own lifecycle: the return listener, sown at admission, fires the instant both the accepted export (offset 60) and the declared address (offset 44, committed before the Cell was even asked anything) are present — at offset 61; the represent controller answers at 62; its answer lands directly in the vector's own `findings` slot at 63, in the exact field the listener was told, at birth, to expect. Every offset here, and the exact JSON each one carries, is reproduced in the companion technical reference's record of the fourth attestation.

PART 3 OF 7 • THE MINIMAL FRAGMENT

CHAPTER 03

The Minimal Fragment

The smallest construct that runs, and the three ways a new one is authored.

3.1 Overview

SCOPE What are we doing here.

Not a metaphor — a real, tiny, working example

The codebase itself keeps a genuine minimal example, used in a real end-to-end test, and it is worth reading whole before anything more abstract is said about it. Its own comment calls it exactly what it is:

```
// A specification in the machine's own record grammar: one gate knot and
// its emit bind — the smallest figure that runs.
const SPECIFICATION = [
  { kind: "sys.knot.defined", payload: { id: "note.gate", strategy: "deterministic",
    config: { meta: { title: "Note", gloss: "The first note." },
      wind: { collect: [{ as: "note", match_type: "note.given", reduce: "latest",
field: "text" }] } },
    condition: "note ≠ null" } } },
  { kind: "sys.descriptor.defined", payload: { id: "note.publish", subscribesTo: "note.gate",
    actionConfig: { writes: "note.published" } } }
];
```

Two records. One listener (`note.gate`) that waits for a fact of type `note.given` and, the moment it sees one, is satisfied. One doer (`note.publish`) that watches that listener and, the moment it fires, republishes what it heard as `note.published` . Commit the two records; commit one `note.given` fact; the machine, on its own, produces the readiness and then the publication — nothing else was ever declared, and nothing else was needed.

Why this is worth dwelling on

Because every other figure in this book — the four vectors of a press-release study, the seven-level cascade of [Part 2](#), the whole admission ceremony that opens a Cell in [Part 7](#) — is built from nothing more than more pairs like this one, wired to watch each other's output. Nothing about a Cell is

architecturally special; it is a fence drawn around a cluster of these same pairs, with one extra rule (results only leave through one door, once a person agrees). A seven-level cascade is seven of these pairs, each one's doer publishing into the next one's condition. There is no third, more powerful primitive anywhere in the system waiting to be discovered — the minimal fragment is the vocabulary, in full.

The word the codebase uses for this on purpose

The two base pieces this pair is built from are described, in their own source comments, as **degenerate** configurations of one underlying idea, not as two different kinds of thing that happen to look similar: a listener with the simplest possible rule is "the degenerate clew that winds locally and projects nothing... a configuration of the same accumulator, never a different kind of unit"; a doer with no extra conditions attached is "the minimal bind — one activation, no demands, no gates." Degeneracy, here, is named deliberately, not apologised for: "no **process** ⇒ identity service; no **present** ⇒ the rendezvous projects straight through — degenerate configurations of the same unit, never new kinds." Simplicity is not a special case bolted onto a more complex general mechanism; the complex cases are what happen when you stop leaving the extra slots empty.

What this buys a study, practically

Once you have genuinely internalised that a Cell, a cascade, and a two-line test fixture are the same construct at different sizes, learning to read a complicated figure becomes a matter of finding its listeners and its doers and tracing who watches whom — never a matter of learning a new kind of machinery for each new shape of study. 3.2 argues this correspondence is not a coincidence of this one system's design, but the natural shape of any well-posed piece of analytical work, cut down to its irreducible core.

3.2 Analytical Rationale

METHOD How do we define and fill systemic gaps — what a minimal fragment gives a study that a loose prompt does not.

Operationalising a question

Any research method eventually has to answer the same practical question: how small does a sub-question have to be before you can actually act on it? The standard answer is called **operationalising** it — stating, precisely, what would count as this question being answerable, and what happens the moment it is. A knot's **condition** is exactly the first half of that ("this counts as answerable — the fields I need are all present"); a bind is exactly the second half ("and here is what happens the instant

it is"). The minimal fragment of 3.1 is operationalisation made mechanical: you cannot register a listener without saying what would satisfy it, and you cannot pair it with a doer without saying what that doer will actually do.

The gap this fills: "ask the model something" is not a unit of work

It is tempting to treat "send this to a language model and see what comes back" as the basic unit of an AI-assisted study. The minimal fragment insists on a stricter unit underneath that: what exactly are we waiting to have in hand before we ask, and what exactly happens to the answer once it arrives. This matters because a loose prompt has no natural stopping point and no natural next step — it produces text, and a person has to decide, freshly, what to do with it every time. A knot-and-bind pair has both baked in from the moment it is declared: the condition says when enough is in hand to ask, and the bind's declared `writes` says, before any answer exists, exactly what shape and what name the result will carry. The gap between "we chatted about it" and "we have a checkable result" is closed at the point of design, not left to be closed by a person reading the output afterwards.

The programme manual's own five declarations

The system's own guidance for building a new figure — `spec/PROGRAMME_MANUAL.md`, chapter 1 — states this as five things that must be nameable before the first line is written, and they map directly onto the minimal fragment's two pieces:

1. **subject and result contract** — the Charter's `title · purpose · question`, or, for a bare knot-and-bind pair, simply what the bind writes;
2. **sufficiency** — the criteria a result will be judged against, with weights and minimums;
3. **human gates** — which of the decisions along the way stay a person's, never the machine's;
4. **budgets** — stated flatly as "mandatory on a canvas", never assumed unlimited;
5. **obligations** — attribution, reachability, termination: who produced this, can it still be found later, does it definitely end.

Whatever cannot be stated against these five, the manual says, is a **red cell**: it either blocks the new figure from being built at all, or is written down honestly as an open gap rather than quietly assumed away. This is the same discipline 3.1's two-record example satisfies almost trivially (a Charter this simple has one criterion: does a fact of the right type exist) and that 7.3's admission ceremony enforces at full weight for anything larger.

Why "there is no `if`" is a feature here, not a limitation

The manual states plainly that the system has no numeric comparison and no conditional branching in the ordinary programming sense — a condition is only ever "is this present" or "is this absent," joined by and. This sounds restrictive until you notice what it forces: a study cannot smuggle in a judgement call disguised as a threshold buried in code. Every real judgement — is this evidence good enough, does this reading meet the bar — has to be stated as a Charter criterion with a weight and a minimum, checked in the open, by an assessment step anyone can read afterwards (7.3). "No `if`" is the same instinct as "no hidden state" from Part 6, applied to judgement itself: if it matters, it has to be declared where it can be seen.

3.3 Functional Specification

SPECIFICATION Where and how flows form — specifically, how a fragment actually comes to exist, and what stands between "someone wants this" and "the machine will run it."

Source: `spec/PROGRAMME_MANUAL.md` (chapters 0, 2, 12–13), `lib/application/{figure-records,validate-figure,programme-shelf,act-deepen}.ts`, `lib/side/figures/composer-fixture.ts`.

Chapter 0's rule, stated first because everything else follows from it

"A particular approach is only ever a programme, never code."

A **figure** — the specification's word for any named arrangement of knots and binds — is one of four declared forms: a **records programme** (a plain YAML list of registration records, read at the moment it is used, never compiled in); a **blueprint set** (named seats a person or a `deepen` request can open as a Cell — this is Part 2's cascade seats and the press-reader seat of Part 4 both); a **Cell package** (a charter plus a blueprint plus a policy, re-entering admission fresh every time it is reused — never persisted as a running thing); or a **Cell's own accepted export**, when that export happens to carry a blueprint — which is precisely what the composer, below, produces.

Way one — hand-written, on the shelf

The most direct route: a person writes a YAML file under `programmes/`, following the shelf's own declared-record convention (state what it declares, where it lives, who consumes it, what proves it worked). It is read fresh every time it is used — nothing about a programme is cached or compiled

ahead of time — and it enters the log either by name at the front door (the world reads the person's opening words and picks the closest match from an enumerated list) or by direct request. This is how every worked example in [Part 4](#) and [Part 2](#) actually got onto the log.

Way two — built live, by filling slots

The **composer** is a figure whose whole purpose is to let a person assemble a different figure's blueprint without writing YAML at all. It presents four slots — the bind (the new figure's own identity: an id, a title, what it publishes, its threshold), the perception (which channel it listens to and how it winds), the questions (one control question per commit, accumulating — never overwriting each other), and the harvest (a bare trigger: "I am done filling the slots above"). Only once all four are present does the harvest bind fire, folding everything collected into a genuine blueprint — a local, deterministic action (`assemble@1`), not a request to the outside world. The composer is, itself, an ordinary Cell, admitted through the same ceremony as anything else — which means its own output is nothing special either: a blueprint, ready to be presented for admission as a fresh figure like any other.

Way three — a seat, opened for you

`deepen` — "Open a Cell", the gesture [Part 4](#) and [Part 7](#) both walk in detail — takes a named seat from a blueprint set and builds its inner fragment automatically, without any hand-written YAML at all. Depending on the seat, the act constructs either the simplest possible inner figure (one knot waiting for the person's own words, one bind that releases them — `deepenBlueprint`) or a slightly richer one for a reader seat (one knot waiting for a question, some material, and the person's word to proceed; one bind that asks the world with the seat's own fixed instruction and result schema — `readerBlueprint` , the shape behind every Cell in [Part 4](#)'s worked example). Either way, what gets built is still, underneath, nothing but knot-and-bind pairs — the act is simply doing the authoring a person would otherwise have done by hand.

The one probe every route passes through, regardless

All three routes converge on the exact same gate: `validateFigure` . Before any proposed set of records is allowed anywhere near a real session, it is run, whole, through a disposable throwaway session built purely to test it — the very same registration machinery that will judge it for real, run once in private first. An unknown listener strategy, a malformed condition, a duplicated id — all of it fails here, in the machine's own words, and the real session never even sees a record that could have poisoned it. The manual's own name for this discipline is blunt: "the machine judges its own food." And its own honest limit is stated just as plainly: this proves the records are **well-shaped**, never that the study they describe is a **good** one — that is what an assessment, a review, and ultimately a person reading the result are for ([Part 7](#)).

The vocabulary of choices, once you are authoring

The construct catalogue (manual chapter 2) is short on purpose: everything composes from **sequence** and **join** alone. A deterministic gate, when the only question is whether something is present. A semantic accumulation knot, when a judgement needs to ripen through several rounds before it is ready. A **canvas** — several lanes feeding on each other under a shared budget. A journal knot, for a side record that is not itself the study's main thread. **unfold**, for generative fan-out with a closing form declared in advance — deliberately flat only; nested fan-out is refused. And a rising ladder of how heavy an answer's execution is allowed to be: **emit: writes** (say it plainly) rises to **emit: unfold** (say several things, of a declared shape) rises to a full Cell (delegate it, under a brief, to be judged before it counts). Building a new fragment is choosing the lightest rung on that ladder that honestly fits the question — never reaching for a Cell where a plain listener-and-doer pair would do.

3.4 Technical Implementation

IMPLEMENTATION How exactly is this implemented at the program level, and how does a fragment prove itself before it is trusted.

Source: `lib/application/figure-records.ts`, `validate-figure.ts`, `programme-shelf.ts`, `act-deepen.ts`; `lib/core/registers/{register-knot-definition,register-descriptor-definition}.ts`; `lib/side/figures/composer-fixture.ts`.

From records to registration tuples

figureEmissions is the smallest function in this whole chain, and it is worth seeing because everything else in this part ultimately calls it: given a plain list of `{kind, payload}` records — whether typed by hand into YAML, assembled live by the composer, or built programmatically by **deepen** — it does exactly one thing per record: refuse anything that is not one of the two registration kinds, refuse a payload with no string **id**, and otherwise stamp **key: null** (registration always rides the lane-less, ambient channel — no programme's own lane hears it by accident) and hand it back as a ready emission. Every single fragment in this book, however it was authored, passed through this one function on its way into the log.

The probe, precisely

validateFigure is `PROGRAMME_MANUAL.md`'s "the machine judges its own food," as code. It converts the records via **figureEmissions**; refuses an empty list outright; then opens a **disposable, in-memory session** — built purely for this one check, discarded immediately after — and ingests the records into it exactly as a real session would. If the real registration machinery throws (an unknown

strategy, a condition it cannot parse, a duplicate id) the function catches it and returns `{valid: false, error}`, in the machine's own words. Only a clean pass returns `{valid: true, recordIds}`. Nothing about this check is a separate, lighter-weight linter approximating the real rules — it is the real rules, run once on a throwaway copy before the real thing sees the records at all.

Landing in the register file

Once records pass the probe, two functions actually place them: `registerKnotDefinition` stores a listener's definition and, in the same movement, creates its fresh bank of per-key lanes (with one "probe" lane at a null key, present before any real fact arrives, purely so a replay of the journal can reconstruct the listener's birth before its first real use). `registerDescriptorDefinition` stores a doer's definition and indexes it under every knot it cares about, so that when a readiness tuple later names that knot, the topology processor knows exactly which doers to wake. Re-registering an id under either function **replaces** the old definition and clears any accumulated lane state — described plainly in the source as "the machine's only edit," and used nowhere in ordinary operation; a revision is a fresh figure, never a patch to an old one.

The composer's own local action

`assemble@1` — the pure, local function behind the composer's harvest bind (3.3) — is worth naming precisely because it is not a call to the outside world: it takes the four slots' accumulated content (the bind identity, the perception settings, the array of questions gathered one commit at a time, and the harvest trigger) and folds them, deterministically, into a well-formed blueprint — the very same shape `figureEmissions` and `validateFigure` will later check. Composing a new figure by hand, through the UI, and composing one by writing YAML converge on literally the same downstream check; the composer only changes how the blueprint gets written, never what is required of it once written.

The one generic ceremony, reused everywhere

`admissionCeremonyRecords(prefix, accepts, budget, publish?)` is the function version of `composer.yaml`'s eight hand-written records: a charter gate and author, a canonical-charter listener, a blueprint listener, a planner that seeds the Cell once both are present, and an independent evaluator. Every seat opened by `deepen` across this whole book — the four press-release readers of Part 4, every level of Part 2's cascade if opened as a Cell, the composer's own admission — expands from this one function, parameterised only by a namespacing prefix, what the new Cell may hear, and its budget. There is exactly one admission ceremony in the entire codebase; everything that looks like a different one is this same template with different arguments.

Two sizes, one shape, side by side

The toy example from 3.1 and a real fragment from a worked study, shown together, so the correspondence is not just asserted but visible:

```
// the toy – a test fixture, two records
{ id: "note.gate", strategy: "deterministic",
  wind: { collect: [{ as: "note", match_type: "note.given", reduce: "latest", field:
    "text" }] },
  condition: "note ≠ null" }
{ id: "note.publish", subscribesTo: "note.gate", actionConfig: { writes: "note.published" } }

// the real thing – one vector of a press-release study, two records
{ id: "press.facts.wanted", strategy: "deterministic",
  wind: { collect: [
    { as: "question", match_type: "press.vector.given", reduce: "latest", field: "text",
    where: [{ field: "vector", equals: "facts" }] },
    { as: "findings", match_type: "press.findings.given", reduce: "latest", field: "text",
    where: [{ field: "vector", equals: "facts" }] }
  ] },
  condition: "question ≠ null && findings ≠ null" }
{ id: "press.facts.found",
  operator: { on: "press.facts.wanted", service: { instruction: "Release the vector's question
and findings as a ledger entry.",
  emit: { writes: "press.facts.found" }, action: "release@1" } } }
```

The only real difference is that the second listener waits for two things instead of one, and its doer is an operator bind with a named local action rather than a bare emit descriptor — a slightly richer instance of exactly the same two-record shape, not a different construct. The full journal behind this exact fragment, tuple by tuple, is walked in the sister vault.

PART 4 OF 7 • THE STANDARD ANALYTICAL SCENARIO

CHAPTER 04

The Standard Analytical Scenario

The nine moves common to every study on the shelf, worked in full for one real press-release study.

4.1 Overview

SCOPE What are we doing here.

The whole arc, as a person would actually experience it

1. **State the object and the purpose.** Paste in, or type, whatever the study is actually about — a document, a claim, a question — in your own words, at the front door.
2. **Let the machine propose a method.** It reads what you wrote and names the study method on its shelf that fits best, with its reason stated in plain words, before anything else happens.
3. **Break the object into questions.** The chosen method already knows its own set of angles to look from — a handful of named questions, each waiting for you to actually ask it.
4. **For each question, delegate to an isolated reader.** Type the question, then hand it — with a named brief — to a fenced-off reader built for exactly this kind of question; it already has your material in hand.
5. **Ask, and wait for a structured answer.** One word — "ask the world" — and, a short wait later, a list of findings comes back, each one tagged with its own epistemic status: did the material merely declare this, or does it actually show it happened, or is it genuinely unclear.
6. **Read it, and decide.** Attest — accept it, and it becomes part of the study — or archive it, and it stays as history only, never used.
7. **The finding returns, on its own.** Accepting releases the finding back into exactly the question that asked for it, as a proper entry in the study's own ledger of results — nothing more to do here; the machine carries it.
8. **Once every question has an answer, synthesise — automatically, and not a moment before.** The study's own synthesis step has been waiting, the whole time, for every one of the questions to come back; the instant the last one does, it writes itself.
9. **Arrange the results into a document.** Place the ledger entries and the synthesis onto one page, in whatever order tells the story best — the study's own working document, built entirely from what you chose to place there.

What makes this one arc, not nine separate features

Every move after the first hands off to the next by nothing more than a fact of a particular shape landing somewhere a listener was already waiting for it — exactly [Part 1](#)'s two ingredients, wired end to end. Move 4 is [Part 7](#)'s Cell, opened. Move 7 is [Part 2](#)'s return socket, firing. Move 8 is the same part's barrier, holding until every branch is genuinely in. Nothing in this arc is a special case bolted onto the machine for this one purpose — it is the whole of the rest of this book, walked once, top to bottom, as a person would actually live through it.

Where the person's judgement sits, and where it does not

The machine proposes the method (move 2), extracts and tags findings (move 5), and refuses to synthesise early (move 8) — none of that is a person's job. Deciding whether a finding is actually right (move 6) always is, and stays the one place in the whole arc where nothing proceeds without a human's own reading. [4.2](#) argues this exact division — machine for extraction and structure, person for judgement — is not particular to this one study; it is the standard shape of any responsible use of a delegated reader, whatever the subject.

4.2 Analytical Rationale

METHOD How do we define and fill systemic gaps — and why the worked example in [4.5](#) is an illustration of a pattern, not the pattern itself.

The nine moves, renamed in standard method

move (4.1)	standard analytical name
1. state the object and purpose	scoping
2. the machine proposes a method	method selection, with a stated rationale
3. break into questions	decomposition into sub-questions
4. delegate each to a reader	tasking a controlled sub-task, with a brief
5. ask; get a tagged answer	source-bound extraction, epistemic-status tagging
6. attest or archive	quality review and sign-off
7. the finding returns	release into the study's own record

move (4.1)	standard analytical name
8. synthesise once every branch is in	synthesis under a barrier
9. arrange into a document	write-up / composition

This is, essentially, the shape any competent piece of evidence-based work already follows, whether the "reader" in move 4 is a person, a research assistant, or — as in this book's own worked example — a language model given a narrow brief. Nothing about the mechanism is press-release-specific; the four vectors of 4.5 happen to be which facts, where, what relations, in what form only because that is what suits studying a company's announcement. A different object gets different questions from a different method on the shelf, and every other move stays identical.

Why "any programme on the shelf" is not an empty claim

`PROGRAMME_MANUAL.md` (cited fully in Part 3) names three further study designs built on this same arc, only one of which — the press-release study — has actually been walked live at the time of writing: reading a physical phenomenon by decomposing it into aspects worked under a shared budget; checking a claim against sources with a declared evidence family and a fetch step behind a human gate; and drafting an article by lane, with citations carried by the represent mechanism of Part 2 and versions living as fresh lanes rather than edits. Each of these swaps out what move 3 asks and what move 4's brief says; none of them touches moves 1, 2, 6, 7, 8 or 9 at all — those are the machine's own fixed contract, covered next in 4.3.

What the epistemic-status tag actually buys, and its honest limits

Tagging every finding declared, happened, or unclear forces a distinction most casual reading skips past silently — a company saying it will do something and a company having done something are not the same fact, however similar the two sentences read. This book's own worked example, examined closely in 4.3, shows the tag is not infallible: a reported number can be marked happened as though independently confirmed, when the material only shows the company itself announcing it — a distinction a careful reader has to catch, because the extraction step has no way to know, on its own, whether "the company said X" and "X occurred" are actually the same claim.

4.3 Functional Specification

SPECIFICATION Where and how flows form — the machine's own contract for each of the nine moves, general to any programme on the shelf.

Source: `programmes/intent.yaml`, `programmes/blueprints-reader.yaml`, `spec/ADR-004-playground-demo-stand.md` (d27), `lib/API.md`.

Move 1–2 – the door, fixed regardless of what the study is

Every session, whatever it will become, begins from the identical two-record figure: one listener waiting for the person's opening words, one doer that asks the world to choose, from an enumerated list, the shelf entry closest to what was written – never a free-form choice, always constrained to a declared schema naming every option by name. Whichever programme wins, the same loader sows it the same way, stamping every record it adds with the choosing intention's own id. Nothing about moves 1 and 2 knows or cares what programme is on the shelf; they are, deliberately, the one part of the whole arc that is identical across every study this system could ever run.

Move 3 – decomposition, declared once, at authoring time

A study's own decomposition into questions is nothing more than however many vector-shaped listeners its programme declares – each one waiting for a person's question (discriminated so it cannot be confused with any other question) and, from birth, a **declared return socket** (2.3) naming exactly where a delegated answer must land. This is fixed at the moment the programme is written, not chosen live – a study's shape is visible in full the instant its records are sown, before a single question has actually been typed.

Move 4 – delegation, the same two-ingest act every time

Opening a reader for any one of a study's questions is, without exception, [the same act](#): an admission ceremony first, a return figure sown only once that ceremony's own receipt confirms success. No study on the shelf has its own separate version of this – every delegated question in every programme goes through this identical two-step act.

Move 5 – the reader's fixed brief

A reader seat's brief – what [7.3](#) called the Charter – is declared once, on the shelf, and reused unchanged for every question a study asks through it. The press-reader seat's own instruction and schema, quoted in full because it is the exact vocabulary behind every "declared / happened / unclear" tag anywhere in this book:

"You are an analyst reading a company press release. The material is the release (and any attached text); the question names the vector of interest. Extract findings for that vector only. For each finding mark status declared (the company says it will, plans, positions) or happened (a completed, dated, quantified or signed event) or unclear, and quote the sentence that is the evidence. Do not add facts the material does not contain. Summarise what is established versus what is declared."

```
{ "findings": [ { "claim": "string", "status": "declared | happened | unclear", "evidence":
"string" } ], "summary": "string" }
```

Every reader seat on the shelf carries its own instruction and its own schema in exactly this shape — the content of the brief changes from seat to seat; the fact that a brief exists, fixed, and a structured schema constrains the answer does not.

Move 6 — the fixed lifecycle, covered in full elsewhere

A Cell's confirmation or refusal follows the identical lifecycle whatever the reader's brief was: `running` → `awaiting_assessment` → `ready_for_review` → `sealed` | `archived`, with the human's own `cell.reviewed` decision the only door into either terminal — set out completely in 7.3, and not repeated here because nothing about it varies by study.

Move 7 — return and release, fixed by two rails at once

An accepted finding's journey back into the study is rails 5 and 4 acting together, exactly as described in Part 2: the declared socket receives it, and the study's own vector bind — a plain local action, `release@1` — writes it out as one ledger entry, `{question, findings}`, the instant both halves of its own condition are present.

Move 8 — the barrier, sized to the study's own decomposition

A study's synthesis step demands exactly as many things as the study declared questions in move 3 — four, for the worked example in 4.5; however many a different programme's own decomposition calls for, in general. The barrier mechanism itself (2.1) does not know or care about the number; it simply will not fire until every one of its declared demands has been satisfied at least once.

Move 9 – the document, built from facts the person chose to place

The final arrangement is the companion technical reference's own subject in full detail; the fixed contract behind it is short: placing anything onto the study's working document is one committed fact (`sheet.pinned`, naming what to place and nothing else), removing something is the same fact's opposite, and the document itself is never anything but a fold of exactly those facts, replayed from the log – never a separate, editable page living only in a browser.

4.4 Technical Implementation

IMPLEMENTATION How exactly is this implemented at the program level, and how the same code serves every study on the shelf.

Source: `lib/application/{act-intent,act-deepen,act-apply-data,act-review-cell}.ts`, `lib/application/ingest-register.ts`.

Every move, its act, its wire shape

move	act	wire shape
1-2. the door	<code>actIntent</code>	<code>POST /api/ingest/intent {text}</code>
3. the question	<code>actApplyData</code>	<code>POST /api/ingest/facts {facts: [{factType: "<vector>.given", data}]}</code>
4. delegate	<code>actDeepen</code>	<code>POST /api/ingest/deepen {slot, seat, words}</code>
5. ask	<code>actApplyData</code>	<code>POST /api/ingest/facts {facts: [{factType: "ask.given", data: {level}}]}</code>
6. attest/archive	<code>actReviewCell</code>	<code>POST /api/ingest/cell.review {cellRef, decision}</code>
7. return/release	(no separate call – a direct consequence of move 6's own settle)	–
8. synthesise	(no separate call – the report bind's own barrier firing)	–

move	act	wire shape
9. arrange the document	actApplyData	<pre>POST /api/ingest/facts {facts: [{factType: "sheet.pinned"/"sheet.unpinn ed", data: {ref}, key: null}]}</pre>

Four acts, reused across nine moves — because, as 4.3 set out, moves 7 and 8 are never separately requested at all; they are the automatic downstream consequence of move 6’s single settle, exactly as 1.4’s loop runs to full quiescence before ever handing control back.

Why the same code serves every study on the shelf

Nothing about `actDeepen`, `actApplyData` or `actReviewCell` names the press-release study, or any other study, anywhere in their own source. `actDeepen` is parameterised entirely by which seat is named in the request — the same function builds a press-reader’s inner figure or a cascade level’s inner figure from the identical code path, differing only in which blueprint-building helper the seat’s own declared shape selects (3.3’s “way three”). `actApplyData` does not know what a “vector” is at all; it only ever shapes whatever `{factType, data}` it is handed, refusing anything under a reserved name. `actReviewCell` does not know which Cell it is reviewing came from which study; the lifecycle it enforces (7.3) is identical for every Cell that has ever existed.

This is, concretely, why 4.2’s claim — “any programme on the shelf follows the same nine moves” — is not an assertion taken on faith: the code that runs moves 1, 2, 4, 6, 7, 8 and 9 is genuinely, provably, the same code for every study, because it is written with no knowledge of any particular one. Only moves 3 and 5 — which questions a study asks, and what brief its readers carry — vary at all, and they vary entirely inside a **programme’s own YAML**, never inside the application code that runs it. Swapping in physics for press releases, or fact-checking for either, changes a shelf file; it changes nothing described in this file.

4.5 Worked Reference Implementation

REFERENCE The nine moves of 4.1, with every number real, drawn from session `loop-sem-1` (semantic class, Gemini 2.5 Flash) — reproduced tuple-by-tuple, gesture-by-gesture, in the sister vault. This file is the index into that fuller account, not a repeat of it.

The object

"Acme Corp. today announced record quarterly revenue of \$120M, up 40% year on year, and said it plans to open two new plants in 2027; the CEO called it a turning point."

The nine moves, with their real offsets

move	what happened	offsets
1-2. the door	the text above committed; the world chose the press-release study, reason stated in full	#0–#16
3. four questions	which facts / where the areas / which relations / what form – typed one at a time, across the session	#18, #66, #117, #171
4. delegate ×4	a press reader Cell opened for each – <code>cell@37</code> , <code>cell@87</code> , <code>cell@140</code> , <code>cell@196</code> , each with its own charter	#19–48, 67–98, 118–151, 172–207
5. ask ×4	each Cell's reader answered with tagged findings and a summary	#52, #102, #155, #211
6. attest ×4	Attest · accept on each – every Cell sealed, every export released	#58, #109, #163, #220
7. return + release ×4	each finding carried back into its own vector's slot, released as a ledger entry	#65, #116, #170, #227
8. synthesise	the fourth attestation completed the report bind's barrier; it asked the world and the report was written, unprompted, in the same settle	#228 → #229
9. arrange the document	the four ledger entries and the report pinned to the sheet, in order; the four Cell rows – the workings – removed, leaving only the results	#230–238

Every one of these thirty-odd offsets, its full JSON payload, and the exact function that produced it is walked individually in the sister vault: the companion technical reference walks every one of these offsets in full – the Cell opening, the reading, and the fourth attestation with the report firing.

What the document looked like at the end

NestVM workbench		help new 1		as of 2024-239		Scenario: press area found 1 2 3 4 5		New report Columns	
Attention: intent planned #2 10 Sept 2024, 16:31 progress press release									
Acme Corp. today announced record quarterly revenue of \$120M, up 40% year on year, and said it plans to open two new plants in 2027; the CEO called it a turning point.									
Released on the web: probably you, in the order published from: show the types: intent press area found 1									
Released by: press area found 1									
What reached #1 2 items									
#1 The financial giant is growth engine for world leaders, and which of them are, and what they are, and what									

The intention stands as the head. Below it, four blocks — each a released ledger entry: the vector's own question, a table of claims with their status and their quoted evidence, and the reader's own summary. Last, the report, in its own three parts: what the release shows happened, what the company declares, and what could not be established — the barrier of move 8, made visible as an actual page.

Reading it as an analyst, honestly

The revenue figure and its growth were tagged **happened**; the two plants, **declared** — correct, by the reader's own brief. Two things a careful reader should still catch, because the extraction step has no way to catch them on its own: the CEO calling it "a turning point" was folded into **declared** where a stricter reading would separate out a third category, opinion, since it is neither a plan nor an event; and **happened**, throughout, means the release reports it happened — a company's own announcement of its revenue is not, by itself, an independently audited fact, and nothing in the reader's tagging distinguishes "the company says so" from "this was independently confirmed." Neither of these is a failure of the mechanism described in this book — they are exactly the kind of judgement 4.1's move 6 exists to catch, and exactly why that move stays a person's, never the machine's.

PART 5 OF 7 · PROTOCOLS AND INTEGRATION POINTS

CHAPTER 05

Protocols and Integration Points

The complete outward-facing surface of the system: every ingest type, every projection, every route.

5.1 Overview

SCOPE What are we doing here.

Three verbs, nothing else

Stand outside the machine — a browser, another program, a script — and there are exactly three things you are allowed to ask it:

1. **"Tell me about yourself."** What sessions exist, what shelf of study methods is available, what seats a Cell could be opened from, how this session relates to others. Nothing here changes anything; it is all lookups.
2. **"Here is something new."** A person's opening question, an answer typed into a form, the single word that means "attest this." Every single change the machine ever makes to what it knows arrives through this one road, always as a named kind of change — never as an anonymous blob.
3. **"Show me a computed view."** What is on the sheet right now, what a particular Cell's lifecycle looks like, what a given tuple's whole neighbourhood in the log is. Always a read; never, whatever it looks like on screen, a second copy of the truth living anywhere but the log itself.

That is the entire outward-facing surface of the system. Every button described anywhere in this book — Commit, Ask the world, Open a Cell, Attest, Pin — is, underneath, one call down one of these three roads, with a name attached saying exactly which kind of thing it is doing.

Why this is worth its own part of the book

Because it means learning "what can this system be made to do" is a genuinely finite, listable task — not a matter of discovering emergent behaviour by trial and error. 5.3 simply lists everything currently on offer down road two and road three, in full, because a list that short is actually worth reading whole rather than sampling.

A road for asking, a road for telling, a road for showing – never mixed

Nothing on the "tell me something new" road can ever silently double as a way of reading something back, and nothing on the "show me a view" road can ever have a side effect. This separation is not a style preference; it is load-bearing for everything the rest of the book relies on – [Part 1](#)'s claim that the log is the one truth only holds if there is genuinely no other channel by which something could quietly change it, and [Part 6.2](#)'s formation boundary depends on "show me a view" being, provably, incapable of writing anything back.

5.2 Analytical Rationale

METHOD How do we define and fill systemic gaps – what a genuinely enumerable protocol buys an analytical tool.

"Protocol" in the research sense, made literal

In ordinary research method, a protocol is a fixed procedure written down precisely enough that someone else can follow it and expect the same outcome – a lab procedure, a coding scheme for qualitative analysis, a checklist for how evidence is logged. The word is used loosely by most software as "the API," but here the two meanings genuinely coincide: the literal HTTP contract is the fixed procedure, because everything the system can be made to do is already named, in advance, on the list in [5.3](#) – there is no informal, undocumented way to make it do something else.

The gap: most tools' real capability outruns their documentation

A large enough system, built the ordinary way, tends to accumulate capability faster than anyone writes it down – undocumented parameters, side effects nobody meant to expose, behaviour reachable only by knowing an internal implementation detail. For an analytical platform this is a genuine liability, not just an inconvenience: if you cannot enumerate everything the system might have done to produce a result, you cannot fully audit how that result came to exist. Keeping the outward surface to three roads, and keeping every capability behind them named and registered rather than merely reachable, closes that gap directly – the list in [5.3](#) is not a curated subset of what is possible; the register **is** what is possible, because nothing gets a name without also getting an entry.

Adding a capability is registration, not discovery

Because the surface is closed this way, extending it is a deliberate, visible act: one new row in the ingest register, or one new row in the projection register — never a capability that simply emerges from wiring things together informally. This mirrors, at the level of the whole system's outward face, exactly the same discipline [Part 3](#) applies to one single fragment: nothing exists until it is declared, and the declaration itself is where anyone auditing the system later goes to find out what it can do.

What a closed verb set does not protect against

None of this stops a badly designed capability from being registered and used — a poorly specified act can still be added to the ingest register and still do something unwise. What it protects is narrower and, again, exactly the thing worth protecting: that nothing the system does was ever reachable by an undocumented path. Every effect traces to a named entry someone can read, in full, in [5.3](#) — which is precisely what makes that list worth publishing whole rather than describing only in general terms.

5.3 Functional Specification

SPECIFICATION Where and how flows form — the complete, current list of what the wave API offers, not a sample of it.

Source: [nestvm/lib/API.md](#) (the machine's own protocol reference).

The three roots, in the specification's own words

"The three roots (thin transport skin): `/api/service/ ...` — the harness's REST (sessions, session meta with the core and catalogue versions, the catalogue, the shelf of programmes, a draft-plane probe); `POST /api/ingest/{type}?session=` — the only change of truth, one request cycle per call; `GET /api/projection/{contract}?session=&...` — snapshots. Refusals answer 422 with the machine's own reason."

Every service-root route is a lookup: the list of stored sessions and their watermarks; one session's own meta (its watermark, which machine class is actually mounted, the core and catalogue versions); the whole function catalogue; the shelf of programmes (split into blueprint sets and records programmes) plus a static list of techniques; the seats a Cell could be opened from; a session's lineage (its ancestors and children, [2.3](#)); and one deliberately marked liberty — a draft-plane probe that previews a proposed figure without committing anything, kept for exploratory work only.

The ingest type register, complete — eleven types

Every single named way to change what the machine knows, as of this writing:

type	act	what it does	status
<code>intent</code>	<code>actIntent</code>	the front door: bootstraps a fresh session, commits the person's opening words and the door's own clock, then — in the same request — the world names the closest-fitting study method and the loader sows it	CURRENT
<code>deepen</code>	<code>actDeepen</code>	open a Cell from a declared slot: a seat's charter plus a deterministic inner figure enter admission; once opened, a return figure is sown at root, addressed to carry the eventual export back (2.4)	CURRENT
<code>facts</code>	<code>actApplyData</code>	one or more plain <code>{factType, data}</code> facts — the ordinary fact factories every form and every bare-word button on the surface use	CURRENT
<code>cell.form</code>	<code>actFormStudyCell</code>	the root ceremony figure plus a wanting fact and its presentations — an earlier route into admission, kept for the study-Cell fixture	CURRENT
<code>cell.feed</code>	<code>actFeedCell</code>	presents an additional fragment into an already-open Cell	CURRENT

type	act	what it does	status
cell.review	actReviewCell	the human decision: {cellRef, decision, actor} – the only door into sealed or archived (7.4)	CURRENT
records	actAssembleFigure	registers listener/door records directly from a submitted list – a laboratory liberty, not the paradigm's authoring path	LIBERTY
figure	actComposeSemanticFigure	compiles a whole figure from a plain-language intent – another laboratory liberty	LIBERTY
represent	actRepresent	the reuse move itself, callable directly (2.4) – a committed value re-presented, retyped and addressed as asked	CURRENT
cell.form {ceremony: composer}	actFormStudyCell	seeds the composer Cell specifically (3.3)	CURRENT
blueprint.admit	actAdmitBlueprint	re-enters a blueprint – the composer's own export, or one presented inline – into admission as a fresh Cell, all-or-nothing	CURRENT

Two types are marked **LIBERTY** rather than **CURRENT** on purpose: they exist for exploratory, laboratory-side work – building a figure directly from raw records, or from a plain-language description – and are explicitly not the paradigm's own authoring path, which is always the planner-bind-plus-admission route [Part 3](#) describes. The distinction is stated in the register itself, not left to be inferred.

The projection register, complete – nineteen contracts

Every named computed view the machine currently offers, grouped by what kind of thing it shows:

The study's own state:

contract	shows
<code>sheet@1</code>	the living document — the intention as its head, the person's own pinned rows in the order pinned (Part 4)
<code>work-queue@1</code>	every figure's stage (waiting · open · published · accepted · closed), each knot as a "figure to fill" — what it hears, its declared fields, its condition, verbatim
<code>figure-history@1</code>	one figure's whole dossier: what reached each of its knots, its intentions and their answers, what it released, what it sowed
<code>publications@1</code>	the released rail — every bind publication and every Cell's accepted export, in order, each with its own declared presentation view
<code>cell-lifecycle@1</code>	every Cell's state, its charter, its assessment, its review — the full record behind Part 7
<code>cell-graph@1</code>	the space of Cells as a graph, with provenance on every edge
<code>study-run@1</code>	the older, pre-workbench study reading (a milestone-driven run)
<code>figure-wiring@1</code>	how a figure's knots and binds are actually wired together
<code>figure-pulse@1</code>	a figure's live activity — what has just moved
<code>scene@1</code>	the milestone-driven scene ladder of the original playground surface

Debug and archival — reading the raw log itself, the one sanctioned exception to "never touch the log directly":

contract	shows
<code>trace@1</code>	one committed tuple, by address
<code>tuple-trace@1</code>	one tuple's whole movement: what caused it, what it changed, what it produced, what shares each of its correlation stamps
<code>station-trace@1</code>	the journal read by place (one station's whole sequence) or by actor (one controller's whole exchange history)
<code>wave-journal@1</code>	the raw journal of one ingress wave

contract	shows
snapshot-at	one retained machine snapshot, at an exact quiescence point

Formed at the wire itself, composed from more than one reading:

contract	shows
<code>vitals</code>	the watermark, tuple/knot/bind counts, which machine class is actually mounted, continuation state
<code>blueprints</code>	the shelf's seats plus the static techniques list

Nineteen names, covering everything on screen anywhere in this book — the sheet, the figure index, the released rail, every Cell card, the whole debug board. Nothing shown anywhere is computed by any path outside this list.

5.4 Technical Implementation

IMPLEMENTATION How exactly is this implemented at the program level, and where the standing checks that govern the whole surface are written down.

Source: `lib/entry/handle-request.ts`, `lib/session/create-document-session-host.ts`, `lib/application/catalogue.ts`.

One handler, two skins

Every request — whatever actually sent it — resolves to exactly one function, `handleRequest` in `lib/entry`, described in its own header as "the engine's own transport... a self-contained HTTP surface... any other consumer speaks the same roots." The web application's own routes (`app/api/ingest/[type]`, `app/api/projection/[contract]`, `app/api/service/[...path]`) are a thin second skin over it, with — by its own comment — "no logic of its own": they only translate a Next.js request into the same shape `handleRequest` already expects. This is why 6.4 could trace a gesture through two consecutive K4 layers without anything actually branching between them — one is the general-purpose transport, the other is that transport's Next-specific coat.

The dispatch table, complete

`handleRequest`'s whole job is reading the path's first segment and routing from there:

route	method	what it does
<code>service/sessions</code>	GET	lists every stored session with its watermark
<code>service/sessions/{id}</code>	GET	one session's own meta — watermark, machine class, core and catalogue versions
<code>service/catalogue</code>	GET	the whole function catalogue, below
<code>service/lineage?session=</code>	GET	a session's ancestors and children (2.4)
<code>service/programmes</code>	GET	the shelf, split into blueprint sets and records programmes
<code>service/seats</code>	GET	every seat a slot could open a Cell from
<code>service/probe</code>	POST	the draft-plane liberty — previews, commits nothing
<code>service/...</code> (anything else)	any	refused: <code>"unknown service '...'"</code>
<code>ingest/{type}?session=</code>	POST only	runs the named typed ingest; 405 on any other method
<code>projection/{contract}?session=&...</code>	GET only	serves the named projection; 405 on any other method
any other root	any	refused: <code>"the wave API has three: service · ingest · projection"</code>
(any refusal, anywhere)	—	status 422, the machine's own error text verbatim in the body

Every refusal, notably, carries the machine's own words back to the caller — never a generic failure — which is the wire-level instance of a discipline that runs through this whole book: an honest failure states what actually went wrong, in the system's own vocabulary, rather than hiding behind a blank error code.

The one request cycle, stated exactly

Every ingest, without exception, follows the same fixed sequence, set at what the codebase's own comments call "the human gate" and stated as four principles: no whole-log replay in normal practice; every ingest arrives with states read from persistent storage that completely reflect what the machine lived; full quiescence yields the fragment and the output states of every component; state is touched only through the state protocol – writes atomic with place, tuple, function and value together, reads never cached. In practice: load the session's stored state at its exact watermark; restore a machine from it; settle it fully (1.4's loop, run once); commit the new tuples, the journal, a fresh snapshot, and every affected reading's updated image, together, in one atomic movement guarded by that same watermark; then discard the machine entirely. Nothing is held in memory between one request and the next – the only thing that persists across requests at all is a simple queue per session, ensuring two requests for the same session never run at once.

The catalogue – every named function, versioned

Beyond the ingest and projection registers, one further registry underlies almost everything else in this book: the **catalogue** of every reusable, named function a record is allowed to invoke by name – a knot's strategy, a bind's local action, a discharge station, a controller. Fifteen entries, current as of this writing:

name@version	kind of station	what it does
<code>deterministic@1</code>	knot	collect; test the closed null-guard condition – Part 3's whole vocabulary
<code>semantic_evaluator@1</code>	knot	accumulate deltas; integrate through the world or locally; grade against declared questions – the strategy behind the cascade's L7
<code>publish@1</code>	bind	emit the activation understanding verbatim, unmodified
<code>format@1</code>	bind	fold the understanding into a deterministic text fragment by template
<code>release@1</code>	bind	release the understanding (or the whole scope) as a result envelope – every vector's own ledger release

name@version	kind of station	what it does
assemble@1	bind	fold the composer's filled slots into a target blueprint (3.4)
ceremony@1	discharge	the deterministic local ceremony: charter, package, assessment — no world involved
study-double@1	discharge	deterministic: assembles a Cell seed with no world call
gemini@1	controller	semantic winding through the world
gemini@2	controller	winding, plus schema-bound service answers — the version behind every real answer in Part 4
programme-loader@1	controller	sows a chosen or proposed programme, probed first, stamped with its own provenance
programme-choice-double@1	discharge	deterministic stand-in: answers with the first allowed programme
schema-double@1	discharge	deterministic stand-in: answers any schema-bound request with a self-naming placeholder of the right shape
inference-stub@1	controller	the plain CI winding double
represent@1	controller	the reuse move itself (2.4)

Every entry is **immutable** once registered: a genuinely changed behaviour becomes a new version under the same underlying lineage — `gemini@1` and `gemini@2` share one lineage identity, two frozen versions — and a record may either pin an exact version or take a bare name, which always resolves to the highest version currently on the shelf. Nothing here is ever edited in place; a change is always a new, additional entry, findable beside the one it supersedes.

The rails — a consolidated reference

Several of this book's key mechanisms are each anchored to one numbered principle in the project's own standing checklist — the "thinking rails" checked on every later decision. Three were unpacked at length where they belong (Part 2 for 4, 5 and 6; Part 6.3 for 11); the rest, named here once so the full set is in one place:

rail	in one line	where this book uses it
1	pins and similar sanctioned side-channels are declared families, never a side effect of navigation	Part 4's sheet
2	(no citation found in this project's own sources)	—
3	a button, or an act, is a fact factory: it shapes a fact so declared rules can route it	5.1, Part 3
4	represent — the one reuse move through the world	Part 2
5	a return socket is declared at birth, never wired after the fact	Part 2
6	authoring (opening a Cell) is categorically heavier than ordinary ingress, and must look it	Part 2, Part 4
7	four distinct transitions — ready, completed, accepted, finished — must never visually collapse into one "done"	7.3's lifecycle
8	the released rail holds bind publications only, never a listener's raw wound state	Part 4
9	(cited nowhere in this project's own sources by that literal name, though the standing checklist itself carries a ninth rule)	—
10	(likewise, uncited here by name)	—
11	the formation boundary: information is formed once, core-side; every surface only presents it	Part 6

Rails 2, 9 and 10 are stated plainly as gaps rather than guessed at — the discipline this whole book tries to follow applies to its own reference tables too.

A shape worth noticing, once more

The three roads of 5.1 are, without the project ever naming them this way, one more instance of the same three-phase shape this book keeps meeting at every scale: **ingest** is collection — what may enter, declared in advance; the machine's own settle is accumulation — where the collected material is actually worked; **projection** is publication — the formed result, released under a declared, checkable shape. A single knot does this. A Cell does this. The whole wire protocol, standing back far enough, does it too.

PART 6 OF 7 · SYSTEM ARCHITECTURE AND COMPONENT INTERCONNECTION
CHAPTER 06

System Architecture and Component Interconnection

The ringed architecture the codebase is built from, and one user action traced through every ring.

6.1 Overview

SCOPE What are we doing here.

A different question from "which file does what"

Most codebases answer "how is this organised" with a list of folders and a sentence per folder. That is useful, and this part gives you that list too (6.3). But the codebase behind this book answers a stronger question first, and the stronger question is the more useful one to carry around: **who is allowed to talk to whom.**

Picture the machine as a set of rings around a centre, like a target or a tree's growth rings. The centre is the raw machinery — the one place where the log actually lives and the rules that move it forward. Around that sits a ring that turns raw machinery into something a request can be made of. Around that, a ring that remembers a session between requests. Around that, a ring of named actions — "open a Cell", "attest", "ask a question" — that a caller can actually invoke. Around that, the wire — an HTTP surface anyone can call. And around everything, what a person actually looks at on screen.

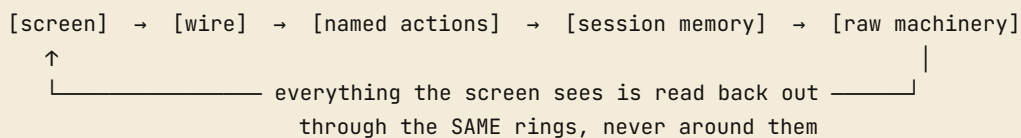
The one rule that matters more than any folder name

A ring may only ever reach its own next-inner neighbour. Not "should, as a matter of style" — the codebase checks this mechanically, by scanning every file's imports (`tests/ring-discipline.test.ts`). The screen never imports the raw log machinery directly. The named actions never import the wire. A ring skipping past its neighbour to grab something two rings in is not a shortcut here; it is a defect the test suite catches before anything ships.

Why this is worth a whole part of the book

Because it is the answer to a question every reader of this book will eventually ask: "if I change how a finding is shown on screen, could that possibly change what the finding actually says?" With rings enforced this way, the answer is a flat no – the ring that shows things can only ever read what an inner ring already decided; it cannot write anything back, and it cannot reach past the reading it was given to touch the log underneath. 6.2 argues this is not incidental tidiness but a requirement of doing analytical work honestly at all.

The picture to hold onto



A gesture goes in from the outside, crosses every ring inward, and its consequences come back out through the same rings, never a shortcut. 6.4 walks exactly one gesture along this whole path with real file names.

6.2 Analytical Rationale

METHOD How do we define and fill systemic gaps.

The gap: presentation drift

Every analytical tool eventually gets asked to change how something looks – a table becomes a chart, a summary gets a new layout, a status badge changes colour. In most systems that change touches the same code paths that computed the thing in the first place, because "compute it" and "show it" were never cleanly separated. Over enough of these changes, a reader can no longer be certain that what changed was the display and not, somewhere quietly, the finding. This is presentation drift, and it is a genuine integrity failure for an analytical system, not a cosmetic one – a report whose numbers might have shifted when someone changed the font is not a report you can stand behind.

The rule that closes it

The machine's own name for the boundary that prevents this is the **formation boundary**: information is formed — decided, computed, folded from the log — strictly on one side; every surface only ever presents what was already formed, and never depends on which surface is attached (`huid 00 $5`, cited throughout the codebase's own comments). Concretely: a **reading** (the machine's word for a computed view of the log — [Part 3](#) and [5.3](#) give this its full technical treatment) is built once, inside the inner rings, and every outer ring — the wire, the screen — can only ever ask for a reading by name and render what comes back. A screen component cannot fold the log itself even if it wanted to; the machinery that does that folding is not on its side of any door it can reach.

What this buys, in standard terms

- **Reproducibility.** Two different screens — the main workbench and the separate debug board described in [Part 4](#) — show the same finding identically, because both ask the same inner ring for the same reading; there is no second code path that could quietly disagree.
- **Auditability of the interface itself, not just the data.** Because the rings are enforced mechanically (a test, not a convention), a reviewer does not have to re-read every screen component to be sure it stayed on its side of the boundary — the suite already checked.
- **Swappability without risk.** The whole screen layer could be replaced — a different UI, a different client entirely — and nothing about what the machine has established would need to change, because the screen never held any of that logic to begin with.

The honest limit

Ring discipline guarantees where logic is allowed to live; it does not by itself guarantee the logic is correct. A reading can still be wrong in what it computes — the rings only guarantee that once it is right, nothing downstream can quietly make it wrong again, and that the same right answer reaches every surface identically. [6.3](#) gives the actual rule set this enforces, ring by ring, and [6.4](#) shows it holding for one real gesture end to end.

6.3 Functional Specification

SPECIFICATION Where and how flows form, and where the key vectors between planes are fixed in the code.

Source: [Lib/TOPLOGY.md](#) ("the axes that hold the spinning machine", working charter v4) and [Lib/README.md](#) (the incubator lid). The machine's own figure for this whole part: a flywheel spins only smoothly while it stays level on fixed axes — four of them, each with a stated purpose, the planes it carries, the files that fix it, and — the part worth reading closely — an explicit list of what it **forbids**.

Axis I — Rotation (the wave axis)

Purpose. Motion made irreversible and countable: every committed tuple is propagated exactly once, in order; a wave runs to full quiescence before anything else happens; nothing overlaps, nothing rewinds. Fixed by the engine's inner loop, the settle loop's run/sweep alternation, and the two independent cursors (1.4) that never move backwards.

Forbids: redelivery to a listener that already saw a tuple; two settles running at once (a caller error, never a queue); delivering a fact to a listener registered after that fact was committed (Vol. 02 §5.1 — the rule behind Part 3's "registration precedes facts"); correlating by position in the log instead of by a named id.

Axis II — Truth (the axle)

Purpose. One axle for all rotation: the append-only log is the only truth. Machine registers, computed readings, stored snapshots — everything else is a cache of a fold over the log, valid only at the exact position it was cut, and discarded the instant it disagrees.

Forbids: editing or renumbering a committed tuple, ever, by anyone; a cache outranking the log (a stale snapshot cannot append — a mismatched watermark is refused at commit, 4.3); tuples travelling inside a state snapshot (state and log are different kinds of thing and must never merge into one).

Axis III — Contours (the radial axis) — the rings of 6.1, named

This is the axis 6.1's picture stands for, and here are its actual names, inside out:

ring (technical name)	directory	what it holds	its one door out
K0 · register file & stations	lib/core/ {wave, registers, topo logy, membrane, cells, runtime, serialise}	the log, the state protocol, the topology processor, CellSpace, the settle loop	lib/core/*/gate.ts per unit
K1 · doors	lib/machine/	the only outside access to K0: createMachine , settleMachine , extra ctMachineState , restoreMachine	lib/machine/gate.ts

ring (technical name)	directory	what it holds	its one door out
K2 · harness	<code>lib/session/</code>	one serialised ingress lane per session; the store ports (wave, machine-state, projection-state)	<code>lib/session/gate.ts</code>
K3 · application	<code>lib/application/</code>	acts (fact factories returning receipts) and readings (named, watermarked snapshot contracts)	<code>lib/application/gate.ts</code> (implied by the ring-discipline guard)
K4 · entry / formation	<code>lib/entry/</code> (the engine's own transport) and <code>huid/formation/</code> (the Next skin over it)	the wave API's three HTTP roots	<code>lib/entry/gate.ts</code> , <code>huid/formation/gate.ts</code>
K5 · surface	<code>huid/modules/</code> , <code>app/</code> route files, <code>debug/</code>	every screen component	(consumes K4 only, over HTTP or its typed wrapper)

Forbids: skipping a ring (application never imports doors or raw registers directly — application only names them via the K1/K2 gates as needed; the surface never imports anything below K4); private closure state anywhere in K0–K3 (all state lives in the one register file, moved only through journalled operations — 1.4); any component reading machine registers directly instead of through a committed tuple; wiring built anywhere but the one declared assembly record.

Axis IV — Observation (the gauge axis)

Purpose. Knowing the spin without touching it. Every way of seeing the machine — a reading, the debug trace, this very region map — is derived, rebuildable, and inert: it measures balance, it never pushes the wheel.

Forbids: an observer influencing anything (a reading emits nothing, registers nothing, and no machine decision may ever depend on one); a silent whole-log re-fold when a reading's stored position cannot be reached from where it is asked — that failure is loud, naming the one explicit recovery path, never a quiet fallback; trusting a model's own self-graded number (the machine recomputes an assessment's aggregate itself — 7.3); an act branching on a reading rather than on its own settle fragment — the one honest gap the charter itself names as "held by review, not yet by a mechanical gauge."

The four-coordinate placement rule

The charter closes with a genuinely useful test for locating any new piece of work: state which turn it belongs to (Axis I), whether it is truth or cache and at what watermark if cache (Axis II), which ring it sits in (Axis III), and whether it moves the machine or only observes it (Axis IV). A piece of work that cannot state all four is not yet placed – and placing it correctly is most of the design decision, not an afterthought to it.

The region map, concretely

`lib/README.md` 's own table, condensed – every directory as one topology coordinate, crossing only through its `gate.ts` :

directory	role
<code>core/wave/</code>	the tuple envelope, the station contracts, the engine's inner loop
<code>core/registers/</code>	the register file and the whole self-journaling state protocol
<code>core/topology/</code>	knot strategies, bind kinds, dispatch, configuration parsing
<code>core/membrane/</code>	the one sweep station – the sole crossing to the world
<code>core/cells/</code>	the Cell protocol, admission, <code>CellSpace</code> (Part 7 in full)
<code>core/runtime/</code>	the settle loop
<code>machine/</code>	the K1 doors
<code>session/</code>	the K2 harness and its three store ports
<code>application/</code>	acts, readings, the declared assembly, the ingest and projection registers
<code>side/world/</code>	controllers – the actual crossings beyond the membrane
<code>side/stores/</code>	store adapters, one per port, one per backend
<code>side/observers/</code>	the reading folds themselves
<code>side/figures/</code>	declared, non-executing P3 content – corpora, fixtures, the composer's own blueprint

directory	role
<code>entry/</code>	K4, the engine's own HTTP transport
<code>huid/formation/</code> , <code>huid/board/</code> , <code>huid/modules/</code>	K4's Next skin and every K5 screen component

6.4 Technical Implementation

IMPLEMENTATION How exactly is this implemented at the program level.

Take **Attest · accept** on a Cell that is `ready_for_review` — the same gesture [Part 7.4](#) examined for what it does to a Cell. This file follows it the other way: not what it does, but which file, in which ring, does each part of it — down through every ring in [6.3's](#) table, and back up.

Down: the request

ring	file	what happens here
K5 · surface	<code>huid/modules/cell-card.tsx</code>	the click handler <code>review("accept")</code> sends <code>POST /api/ingest/cell.review</code> <code>{cellRef, decision: "accept"}</code>
K4 · formation (Next skin)	<code>app/api/ingest/[type]/route.ts</code> → <code>huid/formation/ingest-register.ts</code>	the Next route hands the typed body to <code>handleIngest</code> , which does no logic of its own — it only translates into the engine's own HTTP shape
K4 · entry (the engine's own transport)	<code>lib/entry/handle-request.ts</code>	resolves the path <code>/ingest/cell.review</code> against the ingest type register and calls that entry's <code>run(app, session, body, options)</code>
K3 · application	<code>lib/application/act-review-cell.ts</code> (<code>actReviewCell</code>)	shapes exactly one <code>cell.review</code> record; <code>actor</code> comes from the calling ring's own context, never the request body

ring	file	what happens here
K3 · application	<code>lib/application/ingest-act.ts</code> (<code>ingestAct</code>)	the one write crossing every act shares — hands the record to the session harness, and once it settles, folds the wave's own fragment into every registered reading
K2 · harness	<code>lib/session/create-document-session-host.ts</code>	loads the session's stored state at its watermark, restores a machine from it, asks that machine to settle the new record
K1 · doors	<code>lib/machine/settle-machine.ts</code>	the only outside access into K0 — hands the record and the restored register file to the runtime
K0 · register file & stations	<code>lib/core/runtime/create-wave-runtime.ts</code> → <code>create-wave-engine.ts</code> → <code>create-cell-space.ts</code> → <code>handleReview</code>	the actual work: the settle loop runs the engine, the engine's one top-level processor (CellSpace, wrapping the root topology) routes the fact to <code>handleReview</code> (7.4), which closes the Cell and emits its accepted export; the engine keeps running until every consequence of that — the return figure's readiness, the represent controller's answer, the vector's own readiness, its ledger release — has also settled, because a wave never returns early

By the time K0 hands control back, everything downstream of the attestation has already happened **inside the same settle** — the Cell sealed, the export emitted, the finding returned into its slot, the ledger entry released. Nothing above K0 had to ask for any of that separately; it was one wave, propagated to full quiescence, exactly as [Axis I](#) requires.

Up: the commit, and what the surface learns

K2 commits, in one atomic movement guarded by the session's watermark, four things together: the new tuples, the wave's journal, a fresh station snapshot, and — because `ingestAct` folded them on the way — the **updated images of every reading** that watched this fragment (`cell-lifecycle@1` , `publications@1` , `work-queue@1` , `sheet@1` among them). This last point matters: a reading is not recomputed lazily when the surface later asks for it; it is already advanced and stored at commit time, so the very next read of it is instant.

K3 hands `actReviewCell`'s receipt (`{committed, upToOffset, refs: {cellRef}}`) back up through K4's two skins unchanged, and it lands at K5 as the response to the original `fetch.cell-card.tsx` calls `onCommitted()` — which the workbench page wires to bump a shared refresh key — and every module on the page that depends on that key (the Cell card itself, the released rail, the figure index) fires its own read of the relevant projection immediately, rather than waiting for the standing three-second poll that runs regardless. Both paths end at the same place: `GET /api/projection/cell-lifecycle@1` (or `publications@1`, or `sheet@1`) travels back down through K4 to K3's `readProjection`, which — because the image was already advanced at commit — serves it straight from the stored image, and the response climbs back up to K5 as the Cell card's new state: `sealed`, the accepted export listed under **Released**, the vector's ledger entry now sitting on the rail.

The point of walking it this way

Every one of those ten hops is a single, named, one-directional door — never a shortcut, never a ring reaching two rings deep. That is 6.1's picture made concrete: the gesture that looked like one click on screen is, underneath, a request that only ever moves one ring at a time, all the way to where the log actually is, and a response that only ever moves back the same way.

PART 7 OF 7 · THE CELL — A LOCAL FIELD OF MEANING

CHAPTER 07

The Cell — A Local Field of Meaning

How a delegated sub-investigation is opened, worked, confirmed or refused, and released.

7.1 Overview

SCOPE What are we doing here.

Start from a question you would hand to someone else

Say you are studying a press release and one of your four questions is: "which of these numbers are events that happened, and which are plans?" You could try to answer it yourself, right there in the main document. Or you could do what an analyst normally does with a well-defined sub-question — write it down, hand it with the material to a reader, and wait for a written answer you can check before you use it.

A **Cell** is the second option, built into the machine. It is a separate, walled-off pocket of the same log: its own tiny set of listeners and doers, its own rules about what it may hear and what it may do, and exactly one door out. Nothing that happens inside it is visible to the rest of the study until that door opens — and the door only opens once a person has looked at what came out and said "yes, use this."

What a Cell is not

It is not a different kind of machine. Everything said in [Part 1](#) about the log, listeners and doers is still true inside a Cell — a Cell has its own **knots** (listeners) and **binds** (doers), built from the exact same two ingredients as everything else in the machine. It is not a sandbox in the security sense either — there is no attempt here to stop a hostile actor; it is a sandbox in the editorial sense — a place where a delegated finding stays a draft until someone accepts it.

The four moments this part covers

1. **How one comes to exist.** A Cell does not appear because someone clicked "new Cell". It is admitted, through a fixed, written ceremony: someone proposes a brief (a **Charter** — a title, a purpose, the question, and the criteria it will be judged against), the machine turns that brief plus a

small inner design into a sealed package, and only a package that passes a long list of checks is allowed to open. A rejected proposal leaves no trace at all – nothing half-opens.

2. **How it works while open.** Inside, the Cell's one listener waits for everything it needs (a question, some material, the word to proceed); once it has all three, its one doer asks the world (a language model, in the sessions this book draws on) and gets back a structured answer – a list of findings, each tagged with a status.
3. **How it is confirmed or refused.** The answer is not trusted just because it arrived. An independent check grades it against the Charter's own criteria. Only after that grading does a person get to decide: **Attest** (accept it – the Cell seals, its finding is exported) or **Archive** (keep it as history, but never use it).
4. **Where the results go.** Accepting a Cell releases its finding through exactly one route back into the study – the same slot that asked the question in the first place. Nothing about a Cell's inner workings – its charter, its budget, its raw exchanges with the world – becomes part of the study directly; only that one accepted finding does, and it carries its own provenance wherever it goes.

Why bother with all this ceremony for one question

Because the alternative – letting any sub-task write straight into the study's shared results – is exactly how a study loses track of what it actually knows versus what it merely asked a model. Fencing the sub-task off, writing down its brief, grading its output against that brief, and requiring a human decision before anything leaves is the cost of being able to say, afterwards, precisely where a claim in the final report came from and who signed off on it. The next file argues this is not a NestVM eccentricity – it is standard practice for handling any delegated piece of work, made mechanical.

7.2 Analytical Rationale

METHOD How do we define and fill systemic gaps – what a Cell gives an analytical practice that an undifferentiated log would not.

The standard move this mirrors

Hand a defined sub-task to someone with a written brief; let them work inside their own scope; have an independent reviewer grade the result against the brief before anyone downstream is allowed to rely on it; only then is the finished piece released, carrying its own record of who did it, who checked it and who signed it off. This is not a NestVM invention – it is how a newsroom handles a fact-check, how an audit firm handles a workpaper, how an intelligence shop handles a tasked sub-report. The vocabulary is standard:

standard term	its Cell equivalent
terms of reference / brief	the Charter – title, purpose, the question, and criteria with weights and minimums
scope of engagement	the policy – exactly what the Cell may hear (<code>accepts</code>), what it may do (<code>capabilities</code>), and how much (<code>budget</code>)
the analyst's draft	the Cell's result – the producer bind's one delegated answer
independent quality review	the assessment – a different bind grades the result against the Charter's criteria
partner sign-off / release	attestation – the human's <code>cell.reviewed</code> decision
the released deliverable, with its audit trail	the accepted export – the result in an envelope naming the Charter, the assessment and the review, by their log positions

The gap this fills

An undifferentiated log has no natural way to say "this text came from a controlled sub-task with a stated brief, was graded, and was accepted by a named decision" – without a Cell, a delegated answer is just another fact, indistinguishable in kind from the person's own typed words or from a bind's own local computation. Two systemic gaps a Cell closes:

Gap 1 – provenance of a delegated claim. In most working documents, if you ask "where did this sentence come from", the honest answer is often "somewhere in the chat history, we're not sure exactly which prompt." A Cell makes the question answerable by construction: every accepted finding names its Charter (`charterRef`), the offset of its assessment, and the offset of its review. You are never more than three lookups from the brief that was given and the person who signed off.

Gap 2 – the line between "the model said something" and "the study accepted it". Language models produce text continuously; a study should not. A Cell's lifecycle enforces a hard stop between the world answered (`running` → `awaiting_assessment`) and the study may use this (`sealed`), with a human decision as the only door between them. Nothing about a Cell being open, answered, or even graded 0.9 makes its content usable elsewhere – only the review does.

Decomposition, barriers, and independence — three moves this section keeps meeting

A Cell is where three moves recognisable from ordinary research method become mechanical constructs:

- **Decomposition into a controlled sub-task.** The four vectors of a press-release study ([Part 4](#)) each open their own Cell — the study does not read the whole release once and hope for a coherent answer; it asks one narrow question per Cell, with one brief per question.
- **A barrier before synthesis.** A Cell's own inner report — its evaluator bind — cannot fire until both the Charter and the result exist; the study's outer report cannot fire until all four vectors have returned. Nothing is written up early.
- **Independence of the reviewer.** The machine enforces, not merely recommends, that the bind grading a Cell's result differs from the bind that planned it and the bind that produced it (`assessment: the evaluator must be independent of planner and producer`). A single actor cannot mark its own homework — this is checked, not just written in a style guide.

What is still the analyst's job, not the machine's

The machine grades the existence and shape of a result against the Charter's criteria; it does not grade whether the finding is actually right. In the sessions behind this book the evaluator is a deterministic stand-in that scores any non-empty, well-shaped result 0.9 and an empty one 0.2 — a liveness check, not a fact-check. Reading the findings, catching a status that looks wrong (an announced number marked "happened" when it is really a reported number), noticing a missing quote — that is exactly what attestation is for, and it stays the person's to do, walked through concretely in [4.3](#) and [7.4](#).

7.3 Functional Specification

SPECIFICATION Where and how flows form (the machine's own contract for a Cell), and where the key vectors that move a finding between planes are declared.

Source: `lib/core/cells/cell-protocol.ts`, `admit-cell-package.ts`, `materialise-cell-records.ts`, `create-cell-space.ts` (NestVM Vol. 12 §2–§9, incubated in this repository per `lib/README.md`).

The Charter

A Charter is four things, nothing more:

```
type SemanticCharter = {
  title: string;
  purpose: string;
  question: string;
  criteria: readonly { id: string; question: string; weight: number; minimum?: number }[];
};
```

It is proposed by a bind (through the world or a deterministic stand-in), then made **canonical** by the machine under a ref of the form `charter@<offset of its proposal>`, with a digest of its exact value. From that point the Charter is addressed by ref; it is never re-sent, and its value can never change under that ref — a new brief is a new ref, not an edit.

The policy — what a Cell may hear, do, and spend

```
type CellPolicy = {
  mode: "trial"; // the only mode this slice implements
  isolation: "snapshot" | "subscribed";
  raw_input: "deny" | "allow";
  accepts: readonly { match_type: string }[]; // the ONLY fact types it may hear from
  outside
  capabilities: readonly string[]; // v1 registry: "inference" only
  budget: { max_records: number; max_emissions: number; max_inference_calls: number };
  publish: { candidate: string; accepted: string }; // the one result path, declared before the
  Cell exists
};
```

This is the terms-of-reference document made executable: it is fixed by the **planner** bind at the moment the Cell is requested, travels into the sealed package unchanged, and the machine checks byte-for-byte that the admitted package's policy is identical to what was declared — a model or a discharge cannot widen its own remit between asking and receiving.

Admission — how a Cell comes to exist, or does not

The sequence is the same eight registration records every time (`admissionCeremonyRecords`, one function, reused by every seat on the shelf): a knot waiting for the wanted Charter, a bind that authors it, a knot waiting for the canonical `charter.defined`, a knot waiting for the presented blueprint, a **planner** bind that — once both are present — asks for a Cell to be seeded under a policy fixed at registration, a knot waiting for the Cell's `cell.result`, and an **evaluator** bind demanding both the Charter and the result.

Seeding itself runs through `admitCellPackage` — a pure function, package in, plan or a list of errors out, **all-or-nothing**: no record is emitted until every check has passed. The checks are organised in ten groups covering, by the code's own count, **seventeen individual rules** (`act-admit-blueprint.ts`):

1. the package's outer shape is closed — no unrecognised field is silently accepted, and it declares `version: 1`;
2. the causal request that asked for this Cell exists, is unconsumed, and matches this seed's offset, key, bind and correlation id;
3. the Charter named by ref exists canonically, its key matches, and its digest matches both the package and the canonical definition;
4. the policy in the package is byte-identical to the policy declared when the Cell was first asked for;
5. the blueprint declares version 1, with no duplicate local ids and none using a reserved form (`cell@...`, `...::...`);
6. no listener or bind names a knot that is not local to the blueprint;
7. no local bind writes a protected lifecycle, review or export type;
8. exactly one bind is named as the producer of the declared result type, and it actually writes that type;
9. every fact type a local listener collects is either a reserved protocol type, one of the policy's declared `accepts`, or produced locally — nothing is heard from nowhere;
10. the blueprint's record count fits inside `max_records`.

Only a package that survives all of this is admitted — and admission is atomic with materialisation: local names are qualified (`press.reader.gate` becomes `cell@37::press.reader.gate`), every record is stamped `home` (which Cell it belongs to) and `emittedBy` (the planner's correlation id), and the Cell's `cell.opened` fact, its two records and its `cell.context` are written in that order — registration always lands before the first fact that could reach it.

The lifecycle — four verdicts that never collapse

```
running → awaiting_assessment → ready_for_review → sealed
                                         → archived
                                   (any point, over budget or ceiling) → failed
```

The machine's own comment states the discipline plainly: "admission ≠ runtime success ≠ assessment ≠ human usefulness" — four distinct verdicts, four distinct facts (`cell.opened`, `cell.result`, `cell.assessed`, `cell.reviewed` / `cell.sealed`), never folded into one record. `running` becomes `awaiting_assessment` the moment the declared producer bind's answer lands. `awaiting_assessment` becomes `ready_for_review` only once an independent evaluator (checked mechanically: it may be neither the planner nor the producer) has graded every one of the Charter's criteria, with the aggregate **recomputed by the machine from the Charter's own weights** — a model-

supplied aggregate is read and discarded. Only from `ready_for_review` can a human's `cell.reviewed` move the Cell to its one of two human-only terminals, `sealed` (accept) or `archived` (do not use); a premature or repeated review is refused and changes nothing.

Routing – who hears what, and the one door out

Inside a Cell, facts qualified with `:: ( cell@37::press.reader.gate )` route to that Cell's owner **only** – never to root, never to a sibling Cell. Outside, plain public facts route to root first, then to every open, subscribed Cell on the same lane whose declared `accepts` matches the fact's type – which is how the intention and, later, the person's own words reach a Cell's intake without being addressed to it by name.

Content leaves a Cell through exactly one gate: `press.findings.accepted` (or whichever type the policy's `publish.accepted` names) – a fact the machine itself writes, on acceptance, in an envelope that carries `{cellRef, charterRef, assessmentOffset, reviewOffset, result}`. This is the vector this whole book keeps returning to: the **only** authorised path by which anything a Cell produced can re-enter the study proper. A second, earlier-looking door – `cell.result`, public and visible at root the moment the producer bind answers – exists too, but only so the ceremony's own evaluator can grade the draft; it carries the raw content, but nothing in the study's declared vocabulary (a vector's `findings` slot, in the worked example) is ever filled from it. Only the accepted export, reached through **represent** (rail 4, the reuse move covered in 2.3), can fill a declared slot – which is exactly how the machine keeps "the evaluator peeked at a draft" and "the study accepted this finding" from ever being the same event.

7.4 Technical Implementation

IMPLEMENTATION How exactly is this implemented at the program level, and how can a human and the system attest and verify a Cell's work.

Source: `lib/core/cells/create-cell-space.ts`, `admit-cell-package.ts`, `materialise-cell-records.ts`, `lib/core/registers/{open-cell,close-cell,record-cell-result,record-cell-assessment,count-cell-effect}.ts`, `lib/application/act-review-cell.ts`. A fully worked instance of every function below, with real offsets, is walked tuple by tuple in the companion technical reference for Cell `cell@37`.

Admission, function by function

`admitCellPackage` (`lib/core/cells/admit-cell-package.ts`) is the whole of §7.3's seventeen checks, as one pure function: `{seed offset & key, package, matched causal request, matched Charter, parent capability ceiling}` in, `{plan}` or `{errors: string[]}` out – no side effect, nothing written, until the caller has a clean plan.

`materialiseCellRecords` rewrites the checked blueprint's local names into the Cell's own namespace — `press.reader.gate` becomes `cell@37::press.reader.gate` — and stamps every record `home: {cellRef, localId}` and `emittedBy` (the planner's own correlation id), so a replayed log can always say which admission produced which listener.

`openCell` (a register-file op) is the one place a Cell's registers actually land: lifecycle at `running`, every counter at zero, its own nested topology region created empty — inserted into an ordered map that CellSpace's routing later iterates in that same order.

`CellSpace.handleSeed` ties these together inside the routing processor: find the causal request named in the package's own provenance, find the canonical Charter by ref, call `admitCellPackage`, consume the request either way, and on success emit `cell.opened` followed immediately by the materialised records and the `cell.context` fact — in exactly that order, so the very next tuple to be propagated already has somewhere to land.

While it is open

`interceptCellEmissions` is the effect policy made literal: every batch of emissions a Cell's own nested topology produces passes through this function before it is allowed to become part of the log at all. For each emission it calls `countCellEffect` (increment a counter, compare with the policy's ceiling), refuses any of the reserved lifecycle/review/accepted types outright ("generated topology may not write lifecycle, review or accepted types" — a Cell cannot forge its own attestation), and — for anything that would cross the membrane — checks the `inference` capability before letting it through. The first violation of any kind closes the Cell with one honest `cell.failed` and stops there; it does not try to explain further, because by definition nothing more from that Cell can be trusted.

`recordCellResult` moves the Cell from `running` to `awaiting_assessment` the moment the declared producer bind's answer lands — one register write, journalled, nothing more.

`CellSpace.handleAssessmentProposal` is where the machine checks its own quality-control step: is the causal request real and unconsumed; is the Cell actually `awaiting_assessment`; **is the evaluator a different bind from both the planner and the producer** (checked by identity, not by convention); does every graded criterion belong to the Charter, exactly once, with a grade in `[0, 1]` and an evidence offset that is real committed history. Only if every one of those holds does `recordCellAssessment` move the Cell to `ready_for_review`, with the aggregate recomputed — never trusted — from the Charter's own weights.

Confirming or refusing — the one human-only door

`CellSpace.handleReview` is the entire mechanics of attestation. Given a `cell.reviewed` fact: an unknown Cell is refused; a Cell already `sealed` or `archived` is a silent no-op (repeating a settled decision changes nothing); a Cell not yet `ready_for_review` is refused by name, stating the state it is actually in. Only from `ready_for_review` does `decision: "accept"` do anything — and what it

does is two things in one movement: `closeCell` to `sealed`, and the emission of the accepted export itself, in the policy's declared `publish.accepted` type, carrying `{cellRef, charterRef, assessmentOffset, reviewOffset, result}.decision: "archive"` closes the Cell the same way but emits no export at all — an archived Cell's finding never enters the study.

On the surface this whole function is reached by one gesture. `huid/modules/cell-card.tsx` shows the two controls — gold **Attest** · **accept**, ghost **Archive** — only while a Cell's lifecycle reads `ready_for_review`; either one posts `POST /api/ingest/cell.review {cellRef, decision}`, which resolves to `actReviewCell` (`lib/application/act-review-cell.ts`), which shapes exactly one `cell.reviewed {cellRef, decision, actor}` fact and commits it through the one write door (Part 6 traces this whole vertical stack for the same gesture). The `actor` is stamped by the calling ring from its own authenticated context — never read from the request body — so a Cell's review can always be attributed to a real person, not to whatever the client happened to send.

Verification, two kinds

A Cell is checked twice before its finding can be used, by two different means, and the book keeps both distinct on purpose:

- **Machine verification, automatic, on every Cell.** The independence check on the evaluator; the budget and capability interception on every emission; the recomputed aggregate; the refusal of a premature or repeated review. None of this asks whether the finding is true — all of it asks whether the process was followed: a real brief, an independent grader, spend within the declared ceiling, one human decision and no more.
- **Human verification, the analyst's own reading.** Whether the finding is actually right — whether a status is misapplied, a quote loose, a fact missing — is never checked mechanically; it is what **Attest** or **Archive** is for. Part 4.3 shows this concretely on a real reading; a genuine analyst's mis-read (an announced number marked "happened" as though audited, a company's stated opinion marked "declared" instead of separated out as opinion) is caught this way, not by any check above.

One instance, worked in full

Cell `cell@37` in session `loop-sem-1` (semantic class, Gemini 2.5 Flash) ran through every function on this page in order: seeded at offset 37 from a planner's request, opened with two records (`cell@37::press.reader.gate`, `cell@37::press.reader.answer`), its context committed at 41; asked at 49, answered at 52 (`cell.result` at 53); assessed at 56 (`cell.assessed` at 57, aggregate 0.9, recommendation accept); attested by the person at 58 (`cell.sealed` at 59, the accepted export at 60); returned into the vector's own slot at 63, released as the study's ledger entry at 65. Every one of these offsets, its journal, and the exact JSON payload is reproduced in full in the companion technical reference (the Cell-opening, world-reading and attestation records).

APPENDIX A

A

Glossary

Every term this reference relies on, defined once. Terms coined by this reference itself are marked; the remainder are the system's own established vocabulary, each grounded in a real example from the worked session.

Admission (the ceremony)

The fixed sequence by which a Cell comes to exist: a charter is wanted → authored → defined; a package (charter + blueprint + policy + context + provenance) is assembled by a planner bind and the ceremony discharge; the machine checks the package (some seventeen checks) and either opens the Cell — records first, context after — or rejects it registering nothing. The eight ceremony records are sown at the root under the prefix `admit.<slot>--<seat>`.

Assessment

An independent grading of a Cell's result against its Charter's criteria, proposed by an evaluator bind (through the ceremony discharge, deterministic in this session: 0.9 for any non-empty result) and normalised by the machine: the aggregate is recomputed from the Charter's weights, the recommendation is recorded, and the Cell becomes ready for review. The machine assesses; it never decides.

Attestation (review)

The person's committed decision on a Cell that is ready for review: **Attest** · **accept** commits `cell.reviewed {cellRef, decision: accept, actor}` — the Cell seals and exports; **Archive** closes it without export. The actor is stamped by the transport from its context, not taken from the page.

Barrier (rendezvous)

An operator bind with several demands does nothing until a readiness snapshot for every demand is present in its scope; that waiting point is the barrier. **The report** in this session waited at its barrier for the four vectors; each ceremony's evaluator waited for both the charter and the result.

Bind (operator bind, emit bind)

What fires on readiness. An operator bind waits for the readiness of its activation knot and of every knot it demands (its barrier), gathers their snapshots into a scope, and then either runs a local pure action (`release@1` here) or asks the world with an instruction, the scope and a result schema. It fires once per lane (the one-shot latch). An emit bind simply republishes a knot's readiness snapshot as a fact of a declared type, every time.

Blueprint

The inner topology of a Cell as records: its listeners, its binds, the declared result path (which bind writes which type). For a reader seat the blueprint is one intake listener and one answering bind. Presented to the admission ceremony, checked, then materialised with qualified names.

Cell

An isolated working space opened for one delegated task: it has its own listeners and binds (their names qualified `cell@N::...`), a Charter, a policy (what it may hear, what it may do, budgets), and a lifecycle: `running` → `awaiting_assessment` → `ready_for_review` → `sealed` (or `archived`, or `failed`). Content leaves a Cell only as its accepted export, after the person's review. Facts private to a Cell reach the Cell only; public facts reach the root first and then the Cells whose policy accepts them.

Charter

The written brief of a Cell: a title, a purpose, the question, and criteria (each with a weight and a minimum grade). Proposed by the ceremony from the seat and the person's words, made canonical by the machine under a ref `charter@N` with a digest, and used to grade the result.

Fact (domain.fact)

A tuple of kind `domain.fact` whose payload is `{factType, data}`. The person's inputs, the world's answers and the machine's protocol records are all facts; they differ by type and by who committed them. Reserved type families (`sys.`, `cell.`, `charter.`, `service.`, `inference.`, `bind.`) may not be committed through the facts door.

Formation boundary

Not coined here either — the specification's own name (HUID 00 §5) for the rule that information is computed once, on the machine's side, and every surface only ever presents it. Central to [Part 6](#).

Intention id (uid)

The correlation id a bind mints when it fires — `<bindId>#<n>` — carried by its outgoing request and echoed by the answer. Consumers match answers to requests by this id, never by adjacency in the log. It is also the `emittedBy` stamp on records a bind's intention sowed.

Journal

The machine's own record of every register transition it made while propagating or answering a tuple — station, register, action, detail, the function and file that made it, and the after-image. Stored one document per tuple (`journal/<offset>`) and one per ingress wave (`journal/wave-<watermark>`). The vault's log documents are built from it.

Knot (activation knot)

A listener with named slots: for each slot a collection rule says which fact type fills it (with optional `where` clauses on the fact's fields), which field is taken, and whether the newest value replaces (`latest`) or values accumulate (`append`); a condition says when the knot is ready. On readiness the knot commits a `sys.knot.ready` tuple carrying the snapshot of its slots, then (by default) clears them. Examples: `press.facts.wanted` (slots `question`, `findings`), `cell@37::press.reader.gate` (slots `question`, `material`, `ask`).

Lane (key)

Facts travel on a lane named by their `key`. The session's own lane is `loop-sem-1`; listeners accumulate separately per lane. A null key is the ambient lane — used for registration records and for the sheet's pins, which no programme's listener hears.

Ledger entry

The vector bind's release: `press.<vector>.found {question, findings}` — the question the person asked and the findings the attested Cell returned, released by a pure local action as a bind publication. The four ledger entries and the report are the study's results on the released list.

Lens (Focus · Sheet · Log)

The three ways the workbench shows one session: **Focus** — the released list on the left, the figures index on the right, one figure or one release in the centre; **Sheet** — the living document; **Log** → — the debug board with every tuple, its trace and the stations. Switching a lens is navigation (a URL parameter), never a commit.

Line of reasoning

This book's own name for what [Part 2](#) describes: a chain, or a converging set of chains, of knot-and-bind pairs, each one's output being what a later one was waiting for. Not a term the machine's own specification uses — a reader's word for a pattern in the wiring, covered in full in [2.1](#).

Minimal fragment

This book's name for the smallest complete knot-and-bind pair — one listener, one doer, nothing else — following the codebase's own comment calling an example of exactly this shape "the smallest figure that runs." Covered in [Part 3](#).

Offset

The position of a tuple in the log, counted from 0 and dense (no gaps). Offsets are the addresses by which everything refers to everything else: a charter is `charter@32` because its proposal is tuple #32; a Cell is `cell@37` because its seed is #37; the freshness stamp **as of offset N** on the surface means "everything up to position N-1 has been folded in".

Policy and budget

The fixed terms of a Cell, declared by the planner before the Cell exists and copied unchanged into the package: mode `trial`, isolation `subscribed` (public facts on the same lane reach it if their type is in `accepts`), raw input denied, capabilities (`inference` — it may ask the world), budgets (`max_records`, `max_emissions`, `max_inference_calls`), and the publish pair (its candidate result type and its accepted-export type).

Programme and vector

A study protocol written as records (YAML on the shelf, never code). The press-release programme declares four vectors — Facts of interest, Areas of interest, Relations, Form of facts — each a knot with a question slot and a return socket and a bind that releases the vector's ledger entry, and a report bind that waits for all four. The front door's chooser selects a programme by reading the intention; the loader sows it.

Rail

Not coined here — the project's own word for one numbered entry in its standing checklist of design principles, checked on every later decision. 5.4 gathers every rail this book cites into one table.

Re-presentation (represent)

The move by which a committed value re-enters a declared hearing without being retyped: a `represent.requested` fact names the source (a tuple by session and offset, or an export carried in the request) and the target hearing (type, field, where clauses); the represent controller answers with one fact of the target type, the value placed as it is, stamped `represented` with its provenance and correlated to the request. Used twice per vector: to carry the intention into the Cell as material, and to carry the accepted export back into the slot.

Readiness (sys.knot.ready)

The tuple a knot commits when its condition holds: `{knotId, understanding}` where `understanding` is the snapshot of its slots. Readiness is what activates binds; it is committed to the log so the activation chain is fully recorded.

Reading (projection, fold)

Information formed from the log by a fold — a function stepping over tuples with a small state — and served to the surface with the offset up to which it was folded (`asOfOffset`). Readings used by the workbench: `work-queue@1` (figures and their knots), `publications@1` (the released list), `figure-history@1` (what reached each knot), `cell-lifecycle@1`, `sheet@1`, `trace@1` (one tuple by address). Folds are stored as images and advanced at commit time; nothing is computed in the browser.

Registration record

A tuple of kind `sys.knot.defined` (a listener) or `sys.descriptor.defined` (a bind) that registers a piece of topology. Registration precedes the facts that reach it: a listener never receives facts committed before it existed. Records carry a null key.

Result and accepted export

Inside a Cell, the declared producer bind's publication is the Cell's result (recorded as `cell.result`, a public fact). After the person accepts, the machine emits the accepted export — a fact of the policy's accepted type carrying the result in an envelope that names the Cell, the charter, the assessment and the review by offset. This envelope is the only way content leaves a Cell.

Return (the return socket)

A knot may declare, at registration, which of its slots is its return — the slot a delegated answer will fill (`findings` on the vector knots). When a Cell is opened from that slot, the machine sows a return figure at the root that waits for the Cell's accepted export by cell ref and, holding the declared address, republishes it as a request to carry the export into the slot.

Ring, contour

"Ring" is this book's word for what the specification itself calls a **contour** — one layer of Part 6's radial structure (K0 through K5). The two words name the same thing; this book prefers "ring" in its own prose and gives "contour" wherever it quotes the specification directly.

Seat

A row of a blueprint set on the shelf: the parameters of one kind of Cell a slot may open — title, gloss, questions (the Charter's criteria), threshold, what it publishes, and for a reader seat the instruction and schema its inner bind asks the world with. **Press reader** is the seat used in this session.

Service request

The fact an operator bind commits to ask the world: `{bindId, uid, instruction, scope, schema, emit}`. `emit.writes` names the type the answer must have; `emit.cell` (instead) asks for a Cell to be seeded under a fixed policy. Requests are indexed by their uid so that answers can be verified.

The log (the wave log)

The append-only sequence of tuples that is the session's single source of truth. Every gesture on the surface, every answer of the world and every movement of the machine becomes tuples in it; every reading shown on the surface is a fold over it and can be rebuilt from it. In this session the log has 239 tuples (#0–#238).

The sheet and pins

The living document of a session: the intention as its head, then the rows the person pinned, in pin order. A pin is a fact `sheet.pinned {ref}` on the ambient lane (a figure, a release, any tuple, or a Cell); removal is `sheet.unpinned`. The sheet is a fold of these facts, rebuilt from the log on every refresh, never browser-private.

The three roads

This book's plain-English name for the wave API's three roots (`/service`, `/ingest/{type}`, `/projection/{contract}`) — see Part 5 for the machine's own names and the complete lists behind each road.

The world (controllers, the membrane)

Everything outside the machine that answers requests: the language model (through the Gemini controller), the deterministic ceremony discharge, the programme loader, the represent controller. A second reader of the log (the membrane) offers every tuple to each controller once; a controller that claims a tuple answers it with records that re-enter the log, attributed to the tuple they answer.

Tuple

One entry of the log. Every tuple has a `kind` (one of four: `domain.fact`, `sys.knot.defined`, `sys.descriptor.defined`, `sys.knot.ready`), a `key` (the lane it travels on, or null), a `payload`, and an `offset` — its position in the log, assigned when it was appended. Tuples are never edited or removed; the study's whole history is the sequence of tuples.

Watermark (as of offset)

The number of tuples committed — the position the next tuple will take. The surface shows it as **as of offset N**; the session store guards every commit with it (a commit is refused if the stored watermark is not the one the request loaded).

APPENDIX B

B

Sources and References

Every file this reference draws on. Paths are relative to the `coagnes/` directory of the repository; nothing in this document states a fact, quotes a line, or shows a tuple that was not checked directly against one of these at the time of writing.

THE SPECIFICATION AND STANDING LAW

- `spec/PROGRAMME_MANUAL.md` — how a figure is authored; the source for most of [Part 3](#) and [Part 2](#).
- `spec/ADR-004-playground-demo-stand.md` — the working decision record for the workbench; decision **d27** is the press-release study's own landing record, the direct source for [Part 4](#).
- `spec/ADR-003-contour-protocols.md` — acts as fact factories, the projector/formation-boundary rule, the origin of several cited rails.
- `CLAUDE.md` (host root, outside `coagnes/`) — the eleven "Thinking rails," cited throughout as "host rail N."
- `/Users/leonid/vestacore/learning/meta/meta-bind-01.md` (host level) — the collection · accumulation · publication figure named once, in 5.4's closing note.

THE CORE MACHINE (`NESTVM/LIB/`)

- `lib/README.md` — the region map and conformance ledger behind 6.3.
- `lib/TOPOLOGY.md` — the four axes (Rotation, Truth, Contours, Observation) that are [Part 6](#)'s spine.
- `lib/API.md` — the wave API's three roots, the full ingest and projection registers, the catalogue section — the whole of [Part 5](#).
- `lib/core/wave/{wave-tuple, contracts, create-wave-engine}.ts` — the tuple envelope, the three station interfaces, the engine loop.
- `lib/core/runtime/create-wave-runtime.ts` — the settle loop.
- `lib/core/membrane/create-membrane.ts` — the sweep.
- `lib/core/topology/{create-deterministic-knot, create-operator-bind, create-emit-descriptor, rule-matches, parse-deterministic-condition, create-topology}.ts` — knots, binds, dispatch.
- `lib/core/registers/*.ts` — the register file and the whole self-jouralling state protocol (`op-map.ts`, `record-transition.ts`, and every individual op cited in 1.4 and elsewhere).
- `lib/core/cells/{cell-protocol, create-cell-space, admit-cell-package, materialise-cell-records}.ts` — the whole of [Part 7](#).
- `lib/machine/gate.ts` — the K1 doors.
- `lib/entry/{gate, handle-request}.ts` — the K4 transport, 5.4's route table.
- `lib/session/create-document-session-host.ts` — the request cycle, the four state-protocol principles.
- `lib/application/{act-intent, act-deepen, act-apply-data, act-review-cell, act-admit-blueprint, ingest-act, ingest-register, catalogue, figure-records, validate-figure, programme-shelf, read-lineage, log-address}.ts` — the acts and readings cited across every part.
- `lib/side/world/{create-gemini-controller, create-local-ceremony-discharge, create-represent-controller, create-programme-choice-double}.ts` — the controllers.

- `lib/side/figures/composer-fixture.ts` – the composer's own Charter, blueprint, and `admissionCeremonyRecords`.
- `lib/side/observers/{sheet-projector,release-mark}.ts` – the sheet's fold; the release-mark predicate behind `publications@1`.
- `lib/tests/intent-workbench.test.ts` – the codebase's own "smallest figure that runs" example, quoted in 31.

THE PROGRAMME SHELF (`NESTVM/PROGRAMMES/`)

- `intent.yaml` – the front door.
- `press-release.yaml` – the four-vector study behind Part 4.
- `blueprints-reader.yaml` – the reader seats, including the press-reader brief quoted in full in 4.3.
- `epistemic-cascade.yaml` – the seven-level chain behind 2.3.
- `blueprints-epistemic.yaml` – the seven seats' own titles, glosses and questions.
- `composer.yaml` – the composer's root ceremony.
- `reflective-truth.records.json` / `.md` – the manual's own cited exemplar of a minimal declared figure.

THE SURFACE (`NESTVM/HUID/` , `NESTVM/APP/` , `NESTVM/DEBUG/`)

- `huid/formation/{gate,ingest-register,handle-act}.ts` – the Next skin over the K4 transport.
- `huid/board/perspectives.ts` – the workbench's own declared module map, the source for the dock table implicit in Part 6.
- `huid/modules/{cell-card,knot-card}.tsx` – the Attest/Archive controls, Commit and Open a Cell, walked in 6.4 and 7.4.
- `app/(board)/workbench/page.tsx` – the board's own polling and pin-committing code.
- `debug/perspective.ts` – the debug board's declared module map.

THE WORKED SESSION

- `.sessions/loop-sem-1/` (log and journal, in full in the companion technical reference) (`log/` , `journal/`) – the stored session every real offset in this book is drawn from, in full in `../pr09`.

About this edition

This preview edition was assembled from the [book_p09](#) technical reference vault, itself grounded throughout in the live CoAgnes NestVM codebase and one fully walked session ([loop-sem-1](#) , semantic class, Gemini 2.5 Flash, 15 September 2026). A companion vault ([pr09](#)) reproduces that session tuple by tuple for readers who wish to verify any claim in this document directly against the append log.

Prepared with the assistance of Claude (Anthropic). Comments, corrections and requests for the next revision should be directed to the CoAgnes NestVM documentation owners.

CoAgnes / NestVM / ARCH-PREVIEW / v0.9 • 15 September 2026 • Pre-production preview – subject to change